Structural Econometrics and Inverse Reinforcement Learning: Inferring preferences and beliefs from human behavior*

John Rust† Pranjal Rawat‡

June 30, 2025

PRELIMINARY DRAFT: NOT FOR CIRCULATION

Abstract

This article, for *Oxford Research Encyclopedia of Economics and Finance,* summarizes the literature on structural estimation of dynamic discrete choice models and points out close parallels to the rapidly growing literature on inverse reinforcement learning. Both literatures are focused on inferring underlying preferences (and sometimes beliefs) of human decision makers from observations of their states and actions.

*John Rust is grateful for support from his Georgetown University Professorship fund.
†Professor, Georgetown University jf693@georgetown.edu
‡PhD Candidate Georgetown University. pp712@georgetown.edu

1 Introduction

This article surveys methods for inferring preferences from observed behavior over time, focusing on two parallel literatures: *dynamic discrete choice* (DDC) in structural econometrics (SE) and "inverse reinforcement learning" (IRL) in machine learning (ML).[1] Both literatures rely on the presumption that the agents whose preferences we are interested in recovering are *rational* i.e. they take a sequence of actions over time to maximize an internally perceived reward (or "utility") given their subjective beliefs of how their current actions cause or at least influence potentially uncertain future outcomes that affect their rewards. SE labels this assumption *rationality*, while RL papers typically describe *optimal* versus *sub-optimal* behavior and allow for noisy demonstrations (for example, Ermon et al. (2015)). The goals of DDC and IRL are identical: to invert or infer the underlying objective or reward function (and sometimes also their beliefs) that is presumed to govern the behavior of agents from observations of the sequence of states and actions of a sample of decision makers (DMs) over time.

In economics, structural models are used to test and evaluate economic theories, and to make *counterfactual predictions* of the effect of changes in economic policies[2] or other changes in the environment (i.e. the "rules of the game") on behavior and welfare (i.e. who is hurt and who is helped by the change?). Structural models are distinguished from reduced-form models that summarize behavior without inferring preferences. Reduced-form models require historical policy variation for counterfactual predictions and cannot predict effects of novel policies. Structural models, by internalizing the motivations of DMs, can produce plausible "out-of-sample" predictions of the effect of economic policy changes on behavior and welfare even in the absence of past variation.

For example, Denmark has long had one of the world's highest new car tax rates, with changes only to encourage low-emissions vehicles. Gillingham et al. (2022) used a structural model of equilibrium ownership and trading of cars in Denmark to predict the impact of reducing this "registration tax". The model predicts that by reducing the registration tax and increasing the gas tax, it would be possible of Denmark to 1) improve the welfare of its citizens, 2) increase total tax revenue from cars (registration plus gas tax), and 3) reduce total CO_2 pollution. Structural models provide a faster, cheaper way to evaluate policy changes than trial and error, serving as a simulator to "crash test" new policy ideas.

IRL stems from ML and RL advances enabling intelligent agents for complex robotics and autonomous systems tasks. Much of ML's success has been in *supervised learning* where algorithms learn from labeled training data to predict outcomes. However supervised learning requires extensive labeled data and does not naturally handle sequential decision making where actions have long-term consequences. RL trains agents to maximize a reward through trial-and-error interaction with an environment. When a reward function is known RL algorithms can produce behavior that

[1] To conserve space, we use many abbreviations throughout this article. Readers may consult table 1 for a summary of all abbreviations and acronyms.

[2] In this article, "policy" in the economic sense refers to government interventions (tax rates, regulations, etc.), distinct from the RL/MDP usage where a "policy" is a decision rule mapping states to actions.

approximately maximizes the reward without requiring labeled examples. A good example is chess where the reward function is known (1 for a win, -1 for a loss and 0 for a draw). RL has been able to learn superhuman performance via *self-play,* i.e. repeatedly playing the algorithm against itself resulting in performance that exceeds the best human and computerized chess players, see e.g. Silver et al. (2018).[3]

Still, there are many challenging tasks in AI and robotics where it is unclear what reward function should be used by RL to train intelligent behavior. Christiano et al. (2017) provide a concrete example: "suppose that we wanted to use reinforcement learning to train a robot to clean a table or scramble an egg. It's not clear how to construct a suitable reward function, which will need to be a function of the robot's sensors" and the option of "manually programming their behavior has become increasingly challenging and expensive." Osa et al. (2018). IRL circumvents this by inferring a reward function from limited expert demonstrations. RL can then use the inferred reward to train behavior more rapidly and cheaply across scenarios beyond the demonstration data. The result is intelligent behavior consistent with human preferences, including situations outside the training set.

An early practical success was inverse optimal control (IOC), a related approach originated by Kalman (1964). Mombaur et al. (2010) used IOC to transfer natural human locomotion patterns to humanoid robots. They collected motion capture data of humans walking to targets in five scenarios and inferred a cost function minimizing deviation between model and observed paths. This cost function generated natural robot locomotion to new targets, transferring human movement principles without manual specification. More recently, Barnes et al. (2024) used Google Maps trip data to infer travelers' route preferences reflecting trade-offs between traffic, distance, hills, safety, and scenery. They massively scaled IRL to estimate a route preference function (360M parameters, 110M trips) enabling RL to generate recommendations achieving 16-24% improvement in global route quality, and to the best of our knowledge, represents the largest published study or IRL algorithms in a real-world setting to date.[4]

One limitation of classical IRL is that many real tasks are difficult for humans to demonstrate directly. For instance, training a conversational AI assistant requires demonstrating high-quality responses across thousands of diverse topics, an impractical data collection task. Reinforcement learning from human feedback (RLHF) sidesteps the need for full demonstrations by eliciting preference judgments over pairs of candidate responses, fitting a reward model to match those preferences, and then running RL against the learned reward (Christiano et al., 2017; Ouyang et al., 2022). The most impactful application has been aligning large language models like ChatGPT and GPT-4.

[3] Russell and Norvig (2022) note that supervised learning of chess is limited by the "relatively few examples (about 10^8) compared to the space of all possible chess positions (about 10^{40})" whereas RL can learn through self-play without requiring labeled examples.

[4] While 110 million observations may not seem "limited", it is a tiny fraction of all trips navigated using Google Maps. Jeon et al. (2021) cites additional examples where "defining a reward function beforehand is particularly challenging and IRL is simply more pragmatic." These examples include robotic manipulation, autonomous driving, and clinical motion analysis. See Zhao and Liang (2023) who used Adversarial IRL (AIRL) to estimate context-dependent route preferencs using data on trips by taxi drivers in Shanghai.

Ouyang et al. (2022) showed that a 1.3 billion parameter model fine-tuned with RLHF (InstructGPT) was strongly preferred by human evaluators over the much larger 175 billion parameter base model (GPT-3), demonstrating that alignment with human preferences matters more than raw scale.[5]

We note this survey's relation to other work on related topics. Kang et al. (2025) provided mathematical validation that DDC models are equivalent to maximum entropy IRL. Iskhakov et al. (2020) and Igami (2017) provides a conceptual discussion of AI and structural estimation focusing on model-based RL approaches such as AlphaGo. On the ML side, Arora and Doshi (2021) comprehensively survey IRL methods but without connection to econometrics. Extensions to IOC using deep learning are covered in Abbeel (2016) and connections between IOC and IRL are discussed in Azar et al. (2020). van Cranenburgh et al. (2022) survey machine learning for choice modeling, a related but distinct area from DDC structural estimation. Recent surveys on RLHF methods and applications include Kaufmann et al. (2023) and Abdelnabi et al. (2024). For broader context on machine learning in economics, see Athey and Imbens (2019) and Mullainathan and Spiess (2017), their focus is on supervised learning. Applications of RL in economics and finance are surveyed by Charpentier et al. (2021) and Atashbar and Zhao (2023), with emphasis on applied examples. For comprehensive treatments of DDC methods, see Aguirregabiria and Mira (2010) and Keane et al. (2011). The MDP foundations underlying both DDC and IRL are covered in Puterman (1994) and Rust (1994).

The remainder of this article is organized as follows. Section 2 provides background on dynamic programming and Markovian decision processes, the common ancestor of both SE and RL, including value functions, the Bellman equation, and RL algorithms. Section 3 describes DDC estimation methods and the identification problem. Section 4 summarizes IRL methods and their relation to DDC. Section 5 develops the mathematical foundations of reinforcement learning from human feedback and proves its equivalence to DDC and maximum entropy IRL. Section 6 contrasts the two literatures and discusses unsolved problems.

2 Background on DP and MDPs

The SE and RL literatures both build on a body of work that provides a recursive characterization of optimal sequential decision making under uncertainty, commonly referred to as *dynamic programming* the name coined by one of the prominent early contributors to this theory, Bellman (1957). We will briefly review a special class of DP problems known as stationary, infinite horizon Markovian Decision Problems (MDP) introduced by Bellman and Howard (1960) that is the predominant underpinning of SE and RL and we refer the reader to Bertsekas (2017) for a first rate

[5]The RLHF process for InstructGPT followed three steps: (1) supervised fine-tuning on high-quality human demonstrations to create a better baseline, (2) training a reward model on human rankings of model outputs (using the Bradley-Terry preference model), and (3) optimizing the policy via Proximal Policy Optimization (PPO) with KL regularization to prevent the model from deviating too far from the supervised baseline. See Section 5 for the mathematical formulation.

Table 1. List of abbreviations and acronyms

Abbreviation	**Full Name**
AI	Artificial Intelligence
AIRL	Adversarial Inverse Reinforcement Learning
AGI	Artificial General Intelligence
AL	Apprenticeship Learning
BC	Behavioral Cloning
BIRL	Bayesian Inverse Reinforcement Learning
CCP	Conditional Choice Probability
CCS	Conditional Choice Simulator
DDC	Dynamic Discrete Choice
DM	Decision Maker (Agent)
DNN	Deep Neural Network
DP	Dynamic Programming
DSE	Dynamic Structural Estimation
DU	Discounted Utility
EU	Expected Utility
IOC	Inverse Optimal Control
IL	Imitation Learning
IRL	Inverse Reinforcement Learning
MCMC	Markov Chain Monte Carlo
MDP	Markovian Decision Process
ML	Machine Learning
MPE	Markov perfect Equilibrium
MPEC	Mathematical Programming with Equilibrium Constraints
NFXP	Nested Fixed Point
PI	Policy Iteration
RFE	Reduced Form Estimation
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
RTDP	Real Time Dynamic Programming
SA	Successive Approximations
TD	Temporal Difference

treatment of DP and MDPs.[6]

MDPs are defined by 1) a state space S, 2) an action space A, and 3) the objects $\{r, p, \beta\}$, where

[6]Note that DP and MDPs involve the optimization of discounted utility which was introduced by Samuelson (1937) combined with the expected utility concept introduced by von Neumann and Morgenstern (1944) for problems involving uncertainty. Though Samuelson "had concerns about its its descriptive realism, and it was never empirically validated as the appropriate model for intertemporal choice" (Frederick et al. (2002)), and laboratory tests including the famous *Allais Paradox* (see Machina (1987)) constitute evidence against the hypothesis of expected utility maximization, the MDP framework remains highly relevant for modern work in economics and AI and other areas because MDPs are sufficiently flexible to provide good approximations to a huge range of sequential decision problems. Though there is research on "recursive utility" that relaxes the discounted utility and expected utility assumptions, there are unsolved challenges in applying recursive methods to characterize and numerically approximate optimal strategies for "non-separable" problems.

$r(s,a)$ is the *reward or utility function* providing the payoff to a decision maker (DM), $p(s'|s,a)$ is a *transition density* for next period's state when the state is s and action is a and $\beta \in (0,1)$ is a discount factor. To simplify exposition we assume that both S and A are finite sets, with $|S|$ and $|A|$ elements, respectively.[7] The DM is presumed to choose a *policy* π where $\pi(a|s)$ specifies the probability of taking action a in state s.[8] For any $s \in S$ the support of $\pi(\cdot|s)$ is restricted to a subset $A(s) \subset A$ of choices that are feasible for the DM to take in state s. When a policy is *pure* (deterministic) we write $\delta(s) \in A(s)$ for the action it selects in state s. The DM's objective is to choose a policy π to maximize the expected discounted stream of rewards given by

$$V(s) = \max_{\pi} V_\pi(s), \quad \text{where } V_\pi(s) = E_\pi \left\{ \sum_{t=0}^{\infty} \beta^t r(s_t, a_t) | s_0 = s \right\}, \tag{1}$$

where E_π denotes the expectation operator over the Markov process $\{s_t, a_t\}$ implied by the transition probability p and policy π. Bellman (1957), Howard (1960) and others proved that an optimal value function can be defined recursively in terms of what is now known as *Bellman's equation*

$$V(s) = \max_{a \in A(s)} \left[r(s,a) + \beta \sum_{s' \in S} V(s') p(s'|s,a) \right] \equiv \Gamma(V)(s), \tag{2}$$

so V is the fixed point $V = \Gamma(V)$ where the operator $\Gamma(V)$, known as the "Bellman operator", is the function of s implicitly defined by the right hand side of (2). The Bellman operator can also be shown to be a *contraction mapping* (so for any vectors $V, W \in R^{|S|}$ we have $\|\Gamma(V) - \Gamma(W)\| \leq \beta \|V - W\|$ where β is less than 1 and $\|\cdot\|$ is the Euclidean norm). This implies a unique solution V to the Bellman equation exists. The optimal deterministic decision rule can be recovered from V as the choice $a^* \in A(s)$ that attains the maximum in Bellman's equation

$$\delta^*(s) = \underset{a \in A(s)}{argmax} \left[r(s,a) + \beta \sum_{s' \in S} V(s') p(s'|s,a) \right]. \tag{3}$$

Note that equation (3) implies that the optimal policy is generically a pure strategy, i.e. except for rare cases of ties, for each state s there is a unique $a^* \in A(s)$ that maximizes the right hand side of (3). The associated stochastic policy assigns probability one to $\delta^*(s)$, i.e. $\pi^*(a|s) = \mathbf{1}\{a = \delta^*(s)\}$.

An MDP can be considered as a "game against nature" where "nature's move" is encoded in the transition probability $p(s'|s,a)$ and the optimal deterministic strategy δ^* can be considered as a "best reply" to nature. This framework can be extended to dynamic games where there are multiple rational players in addition to nature. In this setting the reward and transition probabilities for

[7] The general theory carries over essentially unchanged if S or A have an infinite number or even a continuum of elements provided certain boundedness and continuity conditions hold. However we will see that very different methods are used to solve MDPs with large state spaces.

[8] The term "policy" is commonly used for the decision rule π in the RL literature, but the term should be distinguished from the term "policy" used in the introduction, i.e. economic policies such as tax rates, regulations and so forth that can affect the decision rule π as we will discuss in section 3.

each player can depend on the actions of their opponents as well as nature. Maskin and Triole (2001) introduced the concept of *Markov Perfect equilibrium* (MPE) which is a generalization of the concept of Nash equilibrium in static games where each player's strategy is a best reply to all of their opponents and nature. A MPE is defined by the solution to a *system of Bellman equations,* one for each player (except nature). Unlike the single Bellman equation in a single agent game against nature, this system of Bellman equations can have multiple solutions, corresponding to multiple MPE. Further, MPE strategies can be mixed whereas the optimal policy in a single agent MDP is generically pure. There are no known algorithms that are guaranteed to find even a single MPE, since unlike the single agent case, the system of Bellman equations is no longer a contraction mapping.[9] We now summarize several standard algorithms for solving the Bellman equation in the single agent setting.

2.1 Successive Approximations

As is well known, any contraction mapping on a Euclidean space has a unique fixed point, and the contraction property guarantees that the method of *successive approximations* i.e. the sequence $\{V^{(k)}\}$ given by $V^{(k)} = \Gamma(V^{(k-1)})$, for $k = 1, 2, \ldots$ starting from any initial starting guess $V^{(0)}$ will converge to the fixed point V. Successive approximations converge to V at a geometric rate in β and can be very slow when β is close to 1.

2.2 Policy Iteration

In many cases, the *policy iteration* (PI) algorithm (Howard, 1960) provides a much faster method for solving the MDP. This algorithm cycles between *policy valuation* and *policy improvement* steps. At iteration k of the PI algorithm there is a candidate deterministic decision rule $\delta^{(k)}(s)$. The policy valuation step calculates the value implied by this decision rule as $V_{\delta^{(k)}} \in R^{|S|}$ as the solution to the linear system

$$V_{\delta^{(k)}} = r_{\delta^{(k)}} + \beta E_{\delta^{(k)}} V_{\delta^{(k)}}, \tag{4}$$

where $r_{\delta^{(k)}} \in R^{|S|}$ is the vector whose s^{th} element is $r(s, \delta^{(k)}(s))$ and $E_{\delta^{(k)}}$ is the $|S| \times |S|$ matrix whose element in row s and column s' is given by $E_{\delta^{(k)}}(s, s') = p(s'|s, \delta^{(k)}(s))$. It is easy to show that for any feasible deterministic rule δ the matrix E_δ is a Markov transition probability matrix with matrix norm $\|E_\delta\| = 1$, which implies that equation (4) has a unique solution given by $V_{\delta^{(k)}} = [I - \beta E_{\delta^{(k)}}]^{-1} r_{\delta^{(k)}}$.[10] Given the value $V_{\delta^{(k)}}$ the policy improvement step calculates a new decision rule $\delta^{(k+1)}$ given by the right hand side of equation (3) except with $V_{\delta^{(k)}}$ substituted for V. If $\delta^{(k+1)} = \delta^{(k)}$ then policy iteration has converged and it is not hard to show that $V_{\delta^{(k)}} = V$, the unique solution to Bellman's equation (2). Puterman and Brumelle (1979) showed that PI

[9] Iskhakov et al. (2015) introduced the *recursive lexicopgraphical search* (RLS) that can provably find all MPE in the subclass of dynamic directional games.

[10] Alternatively we can show that the policy-specific Bellman operator $\Gamma_{\delta^{(k)}}(V)(s) = r(s, \delta^{(k)}(s)) + \beta E_{\delta^{(k)}} V(s)$ in equation (4) is a contraction.

is mathematically equivalent to using Newton's method to find a zero of the function $F(V) = V - \Gamma(V)$. It is well known that Newton's method is quadratically convergent, and in fact for discrete MDPs, PI converges in a finite number of iterations and typically the number of iterations is quite small, regardless of how close β is to 1. The main downside of PI is that for problems where the state space is large, the work required to solve the linear system (4) for $V_{\delta^{(k)}}$ at each policy valuation step involves $O(|S|^3)$ operations. Thus, in the absence of sparsity or other "special structure" for the matrices $E_{\delta^{(k)}}$, alternative algorithms must be relied on for problems where the number of states $|S|$ exceeds several hundred thousand.

2.3 Avoiding the Curse of Dimensionality with function approximation

The computational burden of solving an MDP when the state space is large was recognized early on: Bellman referred to the problem as the *curse of dimensionality.* For example, in problems where s is a continuous vector in R^d, if we approximate Bellman's equation using interpolation with a finite grid N points in each dimension, then $|S| = N^d$ and the total work to solve the MDP is $O(N^{3d})$, which increases exponentially fast in d. Even when the state space S is naturally discrete and finite, $|S|$ can be very large: for example in classic board games such as chess, the number of possible states (board positions) is estimated to be approximately 10^{44}. Thus it is infeasible to use PI or SA to try to solve finite state approximations to MDPs when the true state space S is sufficiently large.

Bellman and Dreyfus (1959) were among the first to recognize that one way to deal with the curse of dimensionality is to approximate the value function parametrically. For example, we could represent it using a linear combination of fixed basis functions or approximate it with a neural network. Let ϕ denote the vector of parameters defining a parametric approximation to V with V_ϕ denoting the value function evaluated at parameter ϕ (e.g. following Benitez-Silva et al. (2000), take $\phi \in \mathbb{R}^{m+1}$ and write $V_\phi(s) = \phi_0 + \sum_{\ell=1}^m \phi_\ell \psi_\ell(s)$ for chosen basis functions ψ_ℓ). Then an alternative way to approximate the solution to Bellman's equation is to convert it into a nonlinear least squares problem where $\hat{\phi}$ is chosen to minimize the squared "Bellman residuals" over a grid of points $(s_1, \ldots, s_J)$ in S

$$\hat{\phi} = \underset{\phi}{argmin} \sum_{j=1}^{J} [V_\phi(s_j) - \Gamma(V_\phi)(s_j)]^2. \tag{5}$$

The implied value function $V_{\hat{\phi}}$ is an approximate solution to Bellman's equation, (2). We can also consider a modification of PI that we call *parametric policy iteration* (PPI), Benitez-Silva et al. (2000), which uses a parametric approximation $V_{\hat{\phi}^{(k)}}$ and finds $\hat{\phi}^{(k)}$ that mimimizes a least squares criterion similar to (5) to approximate the solution $V_{\delta^{(k)}} = r_{\delta^{(k)}} + \beta E_{\delta^{(k)}} V_{\delta^{(k)}}$ to each policy valuation step k, (4). When V_ϕ is approximated as a linear combination of "basis functions", $\hat{\phi}^{(k)}$ can be computed by linear regression rather than a nonlinear regression as in (5) when we replace the nonlinear Bellman operator Γ with the linear operator $E_{\delta^{(k)}}$. Most of the modern methods for solving high dimensional MDPs involve the use of function approximation, either for the value

function or the policy function, or sometimes both simultaneously.

2.4 Reinforcement Learning

This is a class of stochastic iterative algorithms for solving for the value function V and the optimal policy π. A prominent example is *Q-learning* introduced by Watkins (1989) as a "general formal model of an animal's behavioural choices in its environment" and how it improves over time in response to feedback. In this sense Q-learning embodies a type of "learning" though for our purposes it can be viewed as a stochastic algorithm for solving MDPs since "Q" is just the choice-specific value function in an alternative way of expressing the Bellman equation

$$Q(s,a) = r(s,a) + \beta \sum_{s'} V(s')p(s'|s,a), \tag{6}$$

$$V(s) = \max_{a \in A(s)} Q(s,a). \tag{7}$$

Note that by substituting equation (7) into equation (6) we can represent Q as a fixed point $Q = \Lambda(Q)$ to a mapping Λ that can also be shown to be a contraction mapping. $Q(s,a)$ is the value of taking action a in state s and the optimal policy is to choose the action with the highest $Q(s,a)$, so $V(s)$ is the largest Q value in state s. Watkins proposed a stochastic iterative algorithm for updating Q given by

$$Q_{t+1}(s_t,a_t) = Q_t(s_t,a_t) + \alpha_t \left[r(s_t,a_t) + \beta \max_{a \in A(s_t)} [Q_t(\tilde{s}_{t+1},a)] - Q_t(s_t,a_t) \right], \tag{8}$$

where (s_t, a_t) is the state and action taken by the DM at time step t, and $\tilde{s}_{t+1}$ is a random draw from the transition probability $p(s'|s_t,a_t)$. Q-learning is a form of *stochastic approximation* (Robbins and Munro, 1951). For convergence, the step size sequence $\{\alpha_t\}$ must converge to 0, but at a sufficiently slow rate to ensure the stochastic sequence $\{Q_t\}$ converges with probability 1 to $Q = \Lambda(Q)$. As shown by Watkins and Dayan (1992) and Tsitsiklis (1994), this Q is the unique solution to the modified Bellman equations (6) and (7).[11]

Equation (8) does not specify how a_t is chosen: a typical choice is the *greedy policy* $a_t = argmax_{a \in A(s_t)} Q_t(s_t,a)$. However a key condition for the convergence of Q-learning is that every state-action pair (s,a) must be sampled an infinite number of times. This implies the usual exploration/exploitation tradeoff since suboptimal actions must be taken infinitely often for order for Q learning to converge, but this need not happen under a greedy policy. One solution to this problem is to use a *ε-greedy policy*[12] that takes the optimal action a_t with probability $1-\varepsilon$, and with

[11] The conditions on the step size are 1) $\sum_t \alpha_t = \infty$ and 2) $\sum_t \alpha_t^2 < \infty$. Note $\alpha_t = 1/t$ satisfies these conditions.

[12] Greedy improvement steps (pure exploitation) go back to Bellman (1957) and Howard (1960). Watkins (1989) introduced ε-greedy exploration, and Chapter 2 of Sutton and Barto (1998) formalized both notions for a broad RL audience. The ε-greedy rule picks the greedy action with probability $1-\varepsilon$ and otherwise samples uniformly at random; although it does not minimize regret, it remains extremely popular because it is simple and provides sufficient exploration in practice.

probability ε a randomly selected suboptimal action (i.e. an action a with $Q_t(s_t, a) < Q_t(s_t, a_t)$). However unless ε converges to 0 with t, Q_t will converge to a Q that does not satisfy the Bellman equation (7) since an ε-greedy policy is suboptimal.

Haarnoja et al. (2017) introduced *soft-Q learning* building on the work of Ziebart et al. (2008) on *maximum entropy reinforcement learning* that provides an alternative way to generate continuous exploration, ensuring that all actions are explored infinitely often. These result in policies that converge to mixed strategies that are self-consistent in the sense that the corresponding Q satisfy a modified version of the Bellman equation (2) they refer to as the *soft Bellman equation.* The term "maximum entropy" refers to the inclusion of an "entropy regularization" term in the agent's objective function to result in optimal policies that are generically mixed. Specifically, suppose instead of the original objective in equation (1), we assume the agent's objective includes an entropy term, $H(\pi(s)) = -\sum_{a \in A(s)} \pi(a|s) \log(\pi(a|s))$, so the agent maximizes

$$V(s) = \max_{\pi} V_\pi(s), \quad \text{where } V_\pi(s) = E_\pi \left\{ \sum_{t=0}^{\infty} \beta^t [r(s_t, a_t) + \sigma H(\pi(s_t))] | s_0 = s \right\}, \tag{9}$$

where σ is the weight on entropy. Note that when $\sigma = 0$ we obtain the original MDP objective (1) and the optimal π is generically pure and has 0 entropy, but as $\sigma \to \infty$ the agent's optimal policy is simply to choose each feasible action with equal probability, resulting in a π with maximum entropy. The solution V to the modified DP problem (9) turns out to satisfy a "soft Bellman equation" which turns out to be identical to a version of the "smoothed Bellman equation" derived by Rust (1988), see equations (16) and (15) of section 3.

Q-learning is described as *model-free* since, unlike SA or PI, it does not require explicit calculation of expectations using the transition probability $p(s'|s, a)$: instead all that is needed is be able to *simulate new states using p.* In Watkins' example of how an animal learns to adapt and thrive in its environment, it would be the environment that "draws" the successor state s_{t+1} needed to implement the update in Q_t in equation (8). When trained in this manner Q-learning is indeed "model-free" and so are other RL algorithms that interact directly with the environment.[13]

In many situations Q-learning and related RL methods produce policies that are too noisy and converge too slowly to be useful in practical real time applications, especially when the number of states is large. As Jiang and Xie (2024) note, their "decisions can lead to undesirable outcomes, especially at the early stages of learning when the algorithm has little knowledge of the environment. This is not a problem when the environment is a simulator, but can lead to serious consequences when people are part of the environment." As a result RL training is increasingly done in simulation environments with sufficient computer power to rapidly conduct many thousands or millions of training iterations to produce a much better solution than could be achieved from limited real-world experience. However when models are trained using historical data, the distinction between

[13] See Barto et al. (1995) for other examples of model-free algorithms for solving DP problems they refer to as *real time dynamic programming* (RTDP).

"model-free" and "model-based" becomes nuanced, as we discuss in Section 4. If the algorithm uses an explicit estimate of $p(s'|s,a)$ for planning, it is model-based; if it uses raw transition tuples (s,a,s') directly for TD updates, it remains model-free in RL terminology.[14] The ability to simulate from the true $p(s'|s,a)$ (or a good approximation to it) and for sufficient number of iterations, is the key to producing good solutions using RL. In section 4 we return to this issue, showing that when we have sufficient data on state/action transitions from *past behavior* IRL can use this information to learn agents' rewards in a way that is model-free.

Tabular (i.e. finite state/action) Q-learning, (8), is far too slow for MDPs with large state or action spaces. One scalable solution to this problem to use a Deep Q-Network (DQN), which is a deep convolutional neural network with parameters ϕ that parameterizes the Q-function as $Q_\phi(s,a)$ similar to the way V is approximated by V_ϕ in (5). This transforms the problem from iteratively updating Q-values, as in (8) to using stochastic gradient descent to find ϕ that minimizes the mean squared Bellman error $\rho(\phi)$ corresponding to (7) given by

$$\rho(\phi) = \sum_{i=1}^{N} \left[r(s_i,a_i) + \beta \sum_{s'} \max_{a \in A(s_i)} Q_\phi(s',a) p(s'|s_i,a_i) - Q_\phi(s_i,a_i) \right]^2, \tag{10}$$

over a grid of N points $\{(s_i,a_i)\}$. However the problem with directly minimizing $\rho(\phi)$ over ϕ in (10) is that forming a grid is challenging and the transition probability p may be unknown. Mnih et al. (2015) introduced a stochastic iterative procedure that converges to a $\hat{\phi}$ that approximately minimizes the Bellman error $\rho(\phi)$ and applied it to train effective strategies for classic Atari 2600 video games. Similar methods were subsequently used for board games such as chess, Go and Shogu, see Silver et al. (2018). Simulated data were generated using *self-play* where each player makes choices using a trial policy π resulting in a simulated data set $\mathcal{D}$ of (s,a,s') sequences. Then they used stochastic gradient descent to minimize an approximation to the squared Bellman error (10) using the mean squared *temporal difference error*

$$\rho(\phi) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[\left(r(s,a) + \beta \max_{a'} Q_{\hat{\phi}}(s',a') - Q_\phi(s,a) \right)^2 \right], \tag{11}$$

where $\mathbb{E}_{(s,a,s') \sim \mathcal{D}}$ denotes the sample average over the simulated data set $\mathcal{D}$. Note that in (11) $\hat{\phi}$ is a lagged value of the network parameters from previous training iterations. Use of $\hat{\phi}$ instead of ϕ in (11) is motivated by the fact that the combination of off-policy learning, bootstrapping from subsequent estimates, and the use of a flexible nonlinear function approximator (the 'deadly triad') can lead to instability and non-convergence as noted by Sutton and Barto (2020). Two innovations

[14] The use of Q-learning to train chess strategies is an interesting intermediate case. Although the functional form of $p(s'|s,a)$ for chess is unknown since this transition probability depends on the action of the opponent, Q-learning remains "model-free" since it relies on the environment to produce the next state s_{t+1} directly via self-play, without explicitly modeling or using the game's rules to calculate $p(s'|s,a)$. In contrast, an algorithm like AlphaZero that uses Monte Carlo tree search explicitly uses the game rules for planning and would be considered "model-based".

in the DQN avoided these problems. The first was *experience replay*, whereby transitions are stored in a large "replay buffer" $\mathcal{D}$. The network is then trained on mini-batches of transitions sampled uniformly randomly from $\mathcal{D}$, breaking the strong temporal correlations in the sequence of observations (e.g. persistent streaks in certain parts of the state space), thereby stabilizing the updates by conforming more closely to the i.i.d. assumptions underlying most stochastic gradient methods. The second innovation is the use of a separate *target network* with parameters $\hat{\phi}$, so that $Q_{\hat{\phi}}$ is a lagged version of the current network Q_{ϕ}. Keeping the target network's parameters $\hat{\phi}$ fixed for several iterations of the overall stochastic descent algorithm, keeps the optimization target stable, preventing the current network from attempting to converge to a non-stationary objective. Together, these innovations mitigated the deadly triad and enabled Q-learning to scale to very high-dimensional problems that were previously intractable.[15]

Later studies confirmed that minimizing Bellman error in deep RL does not ensure recovery of the optimal value function. Empirical results show that agents can achieve low Bellman error on replay data while learning suboptimal policies due to overfitting and distribution shift Fu et al. (2019). Theoretical work provides explicit examples where many Q-functions satisfy the Bellman equation on sampled data but only one is optimal, and Bellman error can be minimized by poor value functions in the finite data regime Fujimoto et al. (2022). Despite these gaps, deep RL results in policies that exhibit human level and often superhuman levels of peformance in a variety of high dimensional problems where standard methods such as value or policy iteration are infeasible.

3 Structural Estimation of DDCs

The development of new recursive methods for solving DPs and MDPs in the 1950s and 60s sparked a revolution in economics because these tools enabled economists to model sequential decision making under uncertainty, dynamic strategic interactions, and dynamic equilibria in markets which are key features of nearly all economic problems. Starting in the 1960s DP facilitated the development of new dynamic economic theories that showed how expectations of the future affect the current choices of rational forward looking agents, resulting in new dynamic economic theories embodying *rational expectations* Muth (1961). Since this new framework had the flexibility to predict a wide range of behavior, the natural next step was to develop empirical methods for estimating and testing how well these new theories can explain data. This lead to a new area of econometrics that can be called *dynamic structural estimation* (DSE) which differed from existing econometric methods that could be classified as *reduced form estimation* (RFE). The term "structural" refers to

[15] Mnih et al. (2015) demonstrated that a single deep reinforcement learning agent could learn to play a diverse suite of Atari 2600 games directly from raw pixel inputs and game scores. It reused the same neural network architecture and hyperparameters across all tasks. Without any game-specific engineering, the DQN agent achieved human-level performance in several games. Despite this achievement, it should be highlighted that they did not demonstrate that their DQN converged to the optimal value function as the convergence of the approach is still an open question. Proofs of convergence for stochastic RL algorithms involving function approximation do exist: Tsitsiklis and Roy (1997) proved the probability 1 convergence for Temporal Difference (TD) methods in the case where V_{ϕ} is parameterized as a linear combination of basis functions.

the attempt to infer the underlying preferences and beliefs of economic agents rather than simply summarizing their behavior, the main goal of reduced-form econometrics. In terms of the MDP framework, we can consider an agent's decision rule $\pi(a|s)$ to be the "reduced-form model" because it captures their behavior, whereas structural econometrics tries to infer the underlying *structure* i.e. the preferences and beliefs $\{\beta, r, p\}$ as well as the implied optimal decision rule π.[16]

Robert E. Lucas (1976) provide a practical rationale for why we do structural econometrics. He argued that reduced-form econometric models produce misleading predictions of the effect of economic policy changes, whereas structural models that estimate the underlying preferences of consumers and the production technologies of firms are more likely to accurately predict the effects of policy changes. Policy evaluation requires counterfactual predictions of how economic agents and the economy will react to changes in economic policy and the economic environment. To explain Lucas's argument in the MDP framework, let ρ devote a set of environment or policy variables that affect the preferences (or profits) and beliefs of economic agents. We can write the reward function as $r(s, a, \rho)$ and beliefs as $p(s'|s, a, \rho)$ to emphasize the dependence of these quantities on ρ. We can solve the MDP model for different values of ρ to to make counterfactual predictions of how behavior $\pi(a|s, \rho)$ and welfare $V(s, \rho)$ change in response to changes in ρ. As noted, when there is insufficient variation in ρ (or we want to consider changes in ρ beyond the range of historical experience), reduced-form methods cannot provide good estimates of $\pi(a|s, \rho)$ as ρ changes, nor can they predict how it affects welfare. However it is sometimes possible to know how ρ affects $r(s, a, \rho)$ and $p(s'|s, a, \rho)$ even when there has been little or no variation in ρ in the past. The introduction discussed the example of new car taxation in Denmark discussed in the introduction where ρ is the tax rate on new cars. Under the assumption that $r(s, a, \rho)$ additively separable in (s, a) and ρ Gillingham et al. (2022) used structural estimation to recover the component of $r(s, a, \rho)$ that depends only on (s, a), and with this, they could calculate $r(s, a, \rho)$ for any other possible tax rate ρ, due to the their assumption about its functional form. In contrast, we rarely have a good prior known of how ρ affects the decision rule $\pi(a|s, \rho)$ but structural estimation of $r(s, a, \rho)$ and $p(s'|s, a, \rho)$ enables us to calculate $\pi(a|s, \rho)$ for any policy ρ by solving the MDP. These solutions constitute counterfactual predictions of how car holding and trading behavior and welfare changes in response to changes in the new car tax rate ρ.[17]

The earliest work on estimation of dynamic structural models includes Sargent (1978), who esti-

[16]The earliest development of structural econometrics dates back to work at the Cowles Foundation in thee 1940s where econometric methods were developed for inference on *linear simultaneous equation systems* which are static models of market equilibrium. In this work the structure was the coefficients of linear market supply and demand curves, whereas the reduced form were linear regressions that predict how equilibrium prices and quantities (p, q) shift with various variables x that shift the supply and demand curves. This was the earliest work showing how it could be possible to infer the structure (i.e. the coefficients of the supply and demand curves) using only data (p, q, x) on the intersections of the supply and demand curves in different market states x.

[17]Since changes in tax rates ρ aslo affect prices of new and used cars, a counterfactual prediction also has to account for how ρ affects the entire equilibrium in the Danish car market, so both new and used car prices are implicit functions of ρ. These *equilibrium counterfactuals* are also captured by the structural model of Gillingham et al. (2022).

mated the dynamic linear quadratic model[18] of firms' demand for workers by maximum likelihood. In the remainder of this section we provide a selective summary of the literature on structural estimation of DDCs that has the closest connection to the literature on IRL. Key references include Rust (1987), Rust (1988), Rust (1994), Aguirregabiria and Mira (2002), and the survey by Aguirregabiria and Mira (2010).

The goal is to infer the structural objects $\{\beta, r, p\}$ from data on the trajectories of choices and states on a random sample of individuals using a modified version of the finite state and action MDP discussed in section 2. The modification is motivated by the *statistical degeneracy* of the optimal decision rule $\delta^*(s)$ in equation (3), i.e. it is generically a pure strategy (i.e. deterministic function of s). For most choices we actually observe, different individuals will choose different actions in the same observed state s. In order to explain this heterogeneity in behavior, we consider a modified MDP with an augmented state variable $x = (s, \epsilon)$ where the individual observes both s and ε (and thus uses a pure strategy) but we only observe s but not ε, which is a vector of idiosyncratic preference shocks that we do not observe. As a result, from our standpoint individuals appear to be using a mixed strategy. If there is sufficient variation in the unobserved ε component, we can derive a *conditional choice probability* (CCP) $\pi(a|s)$ from the overall (pure) strategy $\pi(a|s, \varepsilon)$ that admits positive probabilities for all feasible choices a.

Building on the contributions of McFadden (1973) for static discrete choice models, we can derive an especially simple expression for the CCP $\pi(a|s)$ given by the *multinomial logit model* or "softmax" function in the RL literature. Suppose the choice sets depend only on s and are finite, and assume that for each s, ε is a vector with the same length as $|A(s)|$ the number of actions, so for $\varepsilon(a)$ is the component of ε associated with action $a \in A(s)$. Assume that the reward for action a in state (s, ε) can be written as $r(s, a) + \varepsilon(a)$. We call this the Additive Separability (AS) assumption. We also assume that the transition density $p(s', \varepsilon'|s, \varepsilon, a)$ factors as $g(\varepsilon'|s')p(s'|s, a)$, for a conditional density $g(\varepsilon'|s')$. We call this the Conditional Independence (CI) assumption. Finally, if we assume ε has a multivariate Gumbel or Type 1 extreme value distribution given by

$$F(\varepsilon|s) = \prod_{a \in A(s)} \exp\{-\exp\{-(\varepsilon(a) - \mu)/\sigma\}\}, \tag{12}$$

where we set $\mu = -\sigma\gamma$ where γ is Euler's constant, then $E\{\varepsilon(a)|s\} = 0$ for $a \in A(s)$. We call this the Extreme Value (EV) assumption. Let $V(s, \varepsilon)$ be the optimal value of this MDP, the solution to

$$V(s, \varepsilon) = \max_{\pi} V_\pi(s, \varepsilon), \quad \text{where } V_\pi(s, \varepsilon) = E_\pi \left\{ \sum_{t=0}^{\infty} \beta^t [r(s_t, a_t) + \varepsilon_t(a_t)] | s_0 = s, \varepsilon_0 = \varepsilon \right\}. \tag{13}$$

Under assumptions AS, CI and EV, we can write the Bellman equation (2) for $V(s, \varepsilon)$ and show it

[18] That is, the law of motion for s_t is a linear autoregression and r is a quadratic function of (s_t, a_t).

has the representation

$$V(s,\varepsilon) = \max_{a\in A(s)} [Q(s,a) + \epsilon(a)], \tag{14}$$

where $Q(s,a)$ is given by

$$Q(s,a) = r(s,a) + \beta \sum_{s'} \sigma \log \left(\sum_{a'\in A(s')} \exp\{Q(s',a')/\sigma\} \right) p(s'|s,a). \tag{15}$$

The derivation of (14) and (15) makes use of an important property of the extreme value distribution for ε, namely that the expectation of $V(s,\varepsilon)$ with respect to ε has an analytical expression given by the so-called log-sum or "smoothed max function" $V(s)$

$$\begin{aligned} V(s) &\equiv E\left\{V(\tilde{s},\tilde{\varepsilon})\,|\tilde{s}=s\right\} = \int_{\varepsilon} \max_{a\in A(s)} \left[Q(s,a)+\varepsilon(a)\right] F(d\varepsilon|s) \\ &= \sigma \log \left(\sum_{a\in A(s)} \exp\{Q(s,a)/\sigma\} \right). \end{aligned} \tag{16}$$

Using equations (15) and (16) we can derive a "smoothed Bellman equation" for V

$$V(s) = \sigma \log \left(\sum_{a\in A(s)} \exp\left\{ [r(s,a) + \beta \sum_{s'} V(s')p(s'|s,a)]/\sigma \right\} \right), \tag{17}$$

Equation (15) defines a "smoothed Q" as the fixed point of the smoothed contraction mapping $Q_\sigma = \Lambda_\sigma(Q_\sigma)$, and (17) defines V as a fixed point of the smoothed Bellman operator (also a contraction mapping) $V_\sigma = \Gamma_\sigma(V_\sigma)$. These operators are equivalent to the corresponding operators for MDP without the state variable ε in equations (6) and (7) and it is not hard to show that Q_σ and V_σ converge to Q and V as $\sigma \downarrow 0$.

The representation of $V(x,\varepsilon)$ in equation (14) means that the decision rule $\pi(a|x,\varepsilon)$ is isomorphic to a static discrete choice model where $Q(s,a)+\varepsilon(a)$ is the utility/payoff from action a. McFadden (1973) showed that the CCP $\pi(a|s)$ is the conditional probability that the DM chooses action $a \in A(s)$ given the observed state s and has the classic multinomial logit formula given by

$$\begin{aligned} \pi(a|s) &= \int I\left\{ Q(s,a)+\varepsilon(a) \geq \max_{a'\in A(s)} [Q(s,a')+\varepsilon(a')] \right\} F(d\varepsilon|s) \\ &= \frac{\exp\{Q(s,a)/\sigma\}}{\sum_{a'\in A(s)} \exp\{Q(s,a')/\sigma\}}. \end{aligned} \tag{18}$$

It is easy to show that as $\sigma \downarrow 0$, $\pi(a|s)$ converges to a degenerate probability that equals 1 if $Q(s,a)$ is the largest Q value and 0 otherwise, i.e. to the optimal pure strategy π in equation (3) for MDPs without the ε variable. If follows that the choice-specific values $Q(s,a)$ in (15) and CCPs $\pi(a|s)$ in (18) are exactly the same as the "soft Q" values and optimal policy derived in the RL

literature under the assumption that agents care about entropy of the policy π, given in equation (9) of section 2.4.

Equation (18) can be considered as the analog of the *policy improvement* step for MDPs in equation (3). Is there an analogous equation for the policy valuation step (4) for regular MDPs where the value for any policy π is the solution to a linear system? Aguirregabiria and Mira (2002) showed that the value V_π corresponding to any CCP π is given by

$$V_\pi(s) = \sum_{a \in A(s)} \pi(a|s)[r(s,a) + E\{\varepsilon(a)|s,a\} + \beta \sum_{s'} V_\pi(s')p(s'|s,a)], \tag{19}$$

where $E\{\varepsilon(a)|s,a\}$ is the conditional expectation of $\varepsilon(a)$ given that action a is chosen in state s under decision rule π. Under the extreme value assumption for ε, we have $E\{\varepsilon(a)|s,a\} = -\sigma \log(\pi(a|s))$. Using this, it is easy to see that the right side hand of (19) equals the smoothed max function defining the smoothed Bellman operator in equation (17). Thus, we can solve the smoothed Bellman equation $V_\sigma = \Gamma_\sigma(V_\sigma)$ using policy iteration just as in the case of MDPs, where we cycle between policy valuation steps (19) and policy improvement steps (18) using the Q_π implied by V_π in equation (15). In fact, just as in the case of MDPs, policy iteration is equivalent to Newton's method for solving the system $V_\sigma - \Gamma_\sigma(V_\sigma) = 0$.

Equation (18) implies that $\sigma \log(\pi(a|s)) = Q(s,a) - V(s) \equiv \mathcal{A}(s,a)$, where $\mathcal{A}(s,a)$ is called the *advantage function* in RL. Using $\mathcal{A}(s,a)$ we can rewrite the nonlinear fixed point equation (15) for Q as a linear system depending on $r(s,a)$ and $\log(\pi(a|s))$,

$$Q(s,a) = r(s,a) + \beta \sum_{s'} \sum_{a' \in A(s')} \left[-\sigma \log(\pi(a'|s')) + Q(s',a')\right] \pi(a'|s')p(s'|s,a). \tag{20}$$

Equation (20) implies that we can decompose Q as $Q(s,a) = Q_r(s,a) + Q_\varepsilon(s,a)$ where $Q_r(s,a)$ is the component of discounted value from observed rewards r and $Q_\varepsilon(s,a)$ is the component from unobserved rewards ε. Further substituting expression (19) in place of (17) in the equation for Q in (15), we can show that Q_r and Q_ε are each the solution to the following linear systems

$$Q_r(s,a) = r(s,a) + \beta \sum_{s'} \sum_{a' \in A(s')} Q_r(s',a')\pi(a'|s')p(s'|s,a) \tag{21}$$

$$Q_\varepsilon(s,a) = \beta \sum_{s'} \sum_{a' \in A(s')} \left[-\sigma \log(\pi(a'|s')) + Q_\varepsilon(s',a')\right] \pi(a'|s')p(s'|s,a). \tag{22}$$

This decomposition can be exploited to reduce the computational burden of estimating DDC models as we will show below.

3.1 Estimation Methods for DDCs

The most common estimation method for DDCs is some form of maximum likelihood. Suppose we observe a random sample of *trajectories* i.e. data on the consecutive states and actions taken by

N individuals $\{(s_{it}, a_{it})|t = 0, \ldots, T_i,\ i = 1, \ldots, N\}$. Suppose that we smoothly parametrize (r, p) using a vector unknown parameters θ, which we denote by r_θ and p_θ.[19] Using the implicit function theorem, we can show that V_θ and Q_θ, the contraction fixed points given in equations (16) and (15), are continuously differentiable function of θ. Using these fixed points and the logit formula for $\pi(a|s)$ in equation (18) we can write a full likelihood function for the data as a function of the unknown parameters θ given by

$$L(\theta) = \prod_{i=1}^{N} \prod_{t=1}^{T_i} \pi_\theta(a_{it}|s_{it}) p_\theta(s_{it}|s_{it-1}, a_{it-1}). \tag{23}$$

The smoothness of V_θ and Q_θ in θ implies that the CCP π_θ and the likelihood $L(\theta)$ are both smooth functions of θ so the standard asymptotic efficiency properties of maximum likelihood apply.

Rust (1988) introduced the Nested Fixed Point algorithm (NFXP) to compute the maximum likelihood estimator (MLE) of θ. It consists of a standard outer hill climbing algorithm to maximize $L(\theta)$ over θ, but at each evaluation of θ it calculates the contraction fixed point $Q_\theta = \Lambda_\theta(Q_\theta)$ that defines the choice-specific values Q_θ where Λ_θ is the operator defined on the right hand side of equation (15). Notice that $L(\theta)$ depends on θ via Q_θ. Su and Judd (2012) proposed an alternative strategy for computing the MLE based on the MPEC algorithm that maximizes a likelihood $L(\theta, Q)$ expressed as a function of (θ, Q), subject to the fixed point constraint $Q_\theta = \Lambda_\theta(Q_\theta)$.[20]

Estimation of DDC models is often done in a two step fashion, where a parametric model of the transition probability $p(s'|s, d)$ is estimated in the first stage using a partial likelihood that does not involve the CCP π in (23), and in the second stage the parameters of r are estimated using another partial likelihood involving only π, treating the estimated parameters of p as if they were known. A final Newton step iteration for maximization of the full likelihood function (23) can be done using the estimated parameters of p and r from steps 1 and 2 as starting values to obtain fully efficient parameter estimates with the correct asymptotic covariance matrix, i.e. it accounts for "first stage" estimation effort in the parameters of p.

The computational burden of NFXP stems from the need to compute the inner fixed point $Q_\theta = \Lambda_\theta(Q_\theta)$ for each trial value of θ in the outer hill-climbing algorithm that maximizes $L(\theta)$. Newton's method (e.g. policy iteration) is typically used to solve for Q_θ (or alternatively for V_θ in equation (19)). Even though Newton's method converges rapidly, it requires $O(|S|^3)$ operations per Newton/policy iteration step, making NFXP burdensome for large state spaces S. Therefore alternative estimation methods have been proposed that reduce the total number of policy iteration steps required to estimate θ.

Hotz and Miller (1993) proposed the "CCP estimator" for r that uses a non-parametric estimate

[19] The discount factor β is often treated as known, though in some cases it is also estimated and thus part of θ.

[20] Su and Judd (2012) claim that the MPEC alogithm is substantially faster than NFXP, but Iskhakov et al. (2016) showed that this was due to using SA to compute the fixed point $Q_\theta = \Lambda_\theta(Q_\theta)$ which is slow when β is close to 1. When Newton's method is used to compute this fixed point, the cpu times for MPEC and NFXP are roughly comparable with NFXP being faster on test problems with large sample sizes.

of $\pi(a|s)$ to avoid repeated solution of Q over the search for θ that NFXP requires. Consider the case where r_θ is linear in parameters, i.e. we can write $r_\theta(s,a) = r(s,a) * \theta \equiv \sum_{k=1}^K r_k(s,a)\theta_k$ for known functions $\vec{r} = (r_1, \ldots, r_K)$ called *features* in the IRL literature. Then (21) implies that $Q_r(s,a) = R(s,a) * \theta$ where R is the solution to

$$R(s,a) = \vec{r}(s,a) + \beta \sum_{s'} \sum_{a' \in A(s')} R(s',a')\pi(a'|s')p(s'|s,a) = \vec{r}(s,a) + \beta ER(s,a), \tag{24}$$

where $ER(s,a)$ is a shorthand for the conditional expectation of $R(s,a)$ the middle of expression in (24). It follows that we only need to solve (24) once using a non-parametric estimate of $\hat{\pi}$ along with Q_ε in (22) to serve as a "correction term" to obtain the following form for the CCP

$$\pi_\theta(a|s) = \frac{\exp\{\hat{R}(s,a) \cdot \theta + \hat{Q}_\varepsilon(s,a)\}}{\sum_{a' \in A(s)} \exp\{\hat{R}(s,a') \cdot \theta + \hat{Q}_\varepsilon(s,a')\}}, \tag{25}$$

where $\hat{R}(s,a)$ is the solution to (24) and $\hat{Q}_\varepsilon$ is the solution to (22) using non-parametric first stage estimators $\hat{\pi}(a|s)$ and $\hat{p}(s'|s,a)$. The CCP estimator $\hat{\theta}$ maximizes the partial likelihood $L_\pi(\theta)$ given by

$$L_\pi(\theta) = \prod_{i=1}^N \prod_{t=1}^{T_i} \pi_\theta(a_{it}|s_{it}), \tag{26}$$

where π is given in (25).[21] Thus, when r is a linear combination of known features, we only need to compute $\hat{R}$ and $\hat{Q}_\varepsilon$ once at the start of the estimation, requiring $K+1$ solutions of systems with $|S|$ equations and unknowns, where K is the number of features. This can be further reduced to a single solution if we assume that $r(s,a_s)$ is known for some "normalizing alternative" $a_s \in A(s)$ as we discuss further below in the section on identification of DDC models.

A cost of the CCP estimator is that it is less efficient than using NFXP to estimate the same partial likelihood criterion (26) due to the noise in the first stage non-paramteric estimates $\hat{\pi}$ that are used to construct the $\hat{Q}_\varepsilon$ function in (25). Aguirregabiria and Mira (2002) (AM) addressed this problem by proposing an iterative estimator called *nested pseudo likelihood* (NPL) that uses an initial non-parametric estimate $\hat{\pi}$ to compute the value function $V_{\hat{\pi}}$ implied by this initial estimate by solving for it in the policy valuation step in equation (19), which we denote as $\hat{V}_{\theta,\hat{\pi}}$ since it depends both on $\hat{\pi}$ and the parameters θ entering the reward function. AM then estimate $\hat{\theta}$ using a pseudo-likelihood similar to (26) except that the CCP $\pi_\theta(a|s)$ is evaluated using (18) using estimated Q functions given by

$$\hat{Q}_\theta(s,a) = r_\theta(s,a) + \beta \sum_{s'} \hat{V}_{\theta,\hat{\pi}}(s'|s,a)p(s'|s,a), \tag{27}$$

[21] See Chernozhukov et al. (2022) for a modified version of the CCP estimator that is "locally robust" i.e. it corrects for sampling error in the first stage estimates of the functions $\hat{H}(s,a)$ and $Q_\varepsilon(s,a)$.

Thus, the likelihood $L_\pi(\theta)$ as depends implicitly on the first stage non-parametric CCP estimate $\hat{\pi}$, which can be regarded as a "nuisance parameter". In principle, estimation noise in $\hat{\pi}$ will contaminate and affect the asymptotic covariance matrix for the parameters of interest θ. However AM established a key *zero Jacobian property* that at the fixed point $\partial V(s)/\partial\pi = 0$ which implies that the first stage nuisance parameter $\hat{\pi}$ is "Neyman orthogonal" with respect to the parameters of interest θ in the sense of Chernozhukov et al. (2022). It follows that the asymptotic covariance matrix for the NPL estimator of θ is unaffected by noise in the first stage $\hat{\pi}$ and is the same as π was known. Further, AM showed that NPL can be iterated: using $\hat{\theta}$, they can compute an updated estimate of the CCPs $\hat{\pi} = \pi_{\hat{\theta}}$) and repeat the policy valuation step to get a new set of Q functions (27), and use these in the likelihood (26) to get an updated estimate $\hat{\theta}$. They refer to NPL as "swapping" the policy iteration and maximization steps of the NFXP estimator (which does maximization in the "outside" loop and policy iteration in the "inside" or nested fixed point loop). Swapping the order of maximization and policy iteration reduces the total number of policy iteration steps required to estimate θ. AM showed that if this iterative version of NPL converges, it produces the same estimate $\hat{\theta}$ that NFXP produces from maximization of the partial likelihood (26). Thus the NPL estimator is as efficient as NFXP but at lower computational cost, and it is more efficient than the CCP estimator but at a higher computational cost.[22]

Bayesian approaches have been proposed for inference in DDC models using Markov Chain Monte Carlo methods that simultaneously simulate the likelihood as well as stochastic simulations to compute estimates of value functions that are similar to "Monte Carlo tree search" used by RL algorithms such as the Q-learning algorithm discussed in section 2.4, see Imai et al. (2009) and Norets (2009). There are also non-Bayesian simulation estimators such as Bajari et al. (2007) that estimate θ as the value that minimizes the degree of suboptimality in observed actions relative to other actions not taken using simulation estimates of Q function evaluated at the actions the DM actually took and other actions not chosen.

3.2 Function Approximation Methods for Large State Spaces

When the state space S is very large (i.e. the state space is continuous or the number of states $|S|$ is in the hundreds of thousands or higher), all of the approaches discussed above that require one or more solutions of large linear systems with $|S|$ equations and unknowns (e.g. NFXP, CCP, NPL, etc.) become computationally infeasible. An alternative approach is to use parametric methods to approximate the value function V as $V_\phi(s) = f(s, \phi)$ where $f(s, \phi)$ is a member of a flexible family such as neural networks where ϕ denote the weights (parameters) defining the network, and then recasting the fixed point problems (15) and (16) as potentially more tractable nonlinear least squares problems. For example we can approximate V as $V_{\hat{\phi}}$ where $\hat{\phi}$ are the network weights that solve the nonlinear least squares problem of minimizing the squared Bellman residuals in equation

[22]See also Dearing and Blevins (2025) who also proposed a similar iterative procedure called *efficient psuedo likelihood* (EPL) that is also an efficient sequential estimator that can estimate parameters of dynamic games.

(5).

$$\rho(\theta, \phi) = \sum_{j=1}^{N} [V_\phi(s_j) - \Gamma_{\theta,\sigma}(V_\phi)(s_j)]^2. \tag{28}$$

Note that the smoothed Bellman operator Γ_σ defined on the right hand side of (16) depends on r and p which in turn are function of the model parameters θ, so we write it as $\Gamma_{\theta,\sigma}$. It follows that the minimizing parameters $\hat{\phi}$ are implicit functions of θ.

Let $\rho(\theta, \phi)$ denote the squared Bellman residuals defined over a grid of N points in the state space. Then a "nested least squares" estimator similar to NFXP can be used estimate θ in large state spaces, where we maximize $L(\theta)$, but in the inner fixed point problem has been transformed into the problem of minimizing $\rho(\theta, \phi)$ over the parameters ϕ for each value of the structural parameters θ. The CCP estimator can also be implemented in a similar fashion where we parametrize R and Q_ε by parameters ϕ_R and ϕ_ε and solve two nonlinear least squares problems to approximate the fixed points in (24) and (22), respectively. Note, that these would only need to solved *once* at the start of estimation, and the resulting solutions $R_{\hat{\phi}_R}$ and $Q_{\hat{\phi}_\varepsilon}$ can then used to evaluate $\pi_\theta(a|s)$ in (25).

Luo and Sang (2024) introduced a penalized likelihood estimator where $\hat{\theta}$ and $\hat{\phi}$ are simultaneously chosen to maximize

$$(\hat{\theta}, \hat{\phi}) = \underset{\theta,\phi}{argmax} \log(L(\theta)) - \lambda\rho(\theta, \phi), \tag{29}$$

where $\lambda \geq 0$ is a penalization parameter. Note that the likelihood depends on both θ and ϕ since both the CCP π and Q are defined in terms of V_ϕ which depends on the network weights, ϕ, in addition to the structural parameters of interest, θ. The consistency of the penalized likelihood estimator requires the use of a *sieve,* i.e the dimension of ϕ must grow to infinity at certain rate with the sample size to guarantee that the error in approximating the true fixed point by V_ϕ converges to zero. As a result they refer to the estimator (29) as SEES, for Sieve-based Efficient Estimator for Structural models.[23]

A drawback of the SEES estimator is that the ϕ parameters determining the value function V_ϕ become nuisance parameters, and for fixed θ the ϕ that maximize (29) $\hat{\phi}$ that minimizes the squared Bellman error $\rho(\theta, \phi)$ will be an implicit function of θ. As a result, computation of the asymptotic covariance matrix for θ requires inverting an analog of an information matrix for (θ, ϕ) jointly which can be challenging to do when ϕ is high dimensional. Nguyen (2025) introduced an estimator called *neural network efficient estimator* (NNES) that maximizes a penalized criterion similar to the SEES estimator (29) but in a sequential fashion similar to the NPL estimator of AM, benefitting from the zero Jacobian property of the NPL estimator and ensuring Neyman orthogonality between the parameters of interest θ and the nusiance parameters ϕ. NNES is initialized from an initial non-parametric estimator of the CCPs, $\hat{\pi}$, but instead of solving for the implied value function V

[23] See Peter Arcidiacono and James (2013) and Kristensen et al. (2021) for other examples of sieve-based estimators for DDC models.

exactly as a system of linear equations (19), it uses a a neural network parameterization of the value function V_ϕ and finds $\hat{\phi}$ that minimizes the sum of squared Bellman residuals $\rho(\theta, \phi)$ in (28). The resulting value function, which also depends on θ (so we write $V_{\theta,\hat{\phi}}$) is then used to form the $\hat{Q}_\theta$ values per (27) which are in turn used to calculated the CCPs $\pi_\theta(a|s)$ and the pseudo likelihood $L_\pi(\theta)$ following the same approach as the NPL estimator. In essence, NNES is a convex combination of the NPL and SEES estimators: 1) like NPL it is iterative, starting from an initial nonparametric estimator $\hat{\pi}$, 2) unlike NPL but similar to SEES, it does not solve for $V_{\hat{\pi}}$ in equation (19) exactly via standard linear algebra, but rather minimizes the square Bellman residuals $\rho(\theta, \phi)$. The main benefit of the NNES estimator over SEES is that the zero Jacobian/Neyman orthogonality property implies that to get the covariance matrix for θ we only have to invert a matrix of the same dimension as θ rather than take the (θ, θ) block of the inverse of a matrix with dimension equal to the sum of the number of parameters in θ and ϕ. For flexible and deep neural networks, the dimension of ϕ can be in the thousands, so this can be a significant advantage.

3.3 The Identification Problem

At its core, structural estimation of DDC models can be regarded as a process of inverting observed behavior to uncover preferences.[24] The observed behavior from the DDC model is captured by the CCP $\pi(a|s)$, and it can be viewed as the "reduced form" model of the agent's behavior since with enough data on (a, s) pairs, we can estimate π nonparametrically without need for further assumptions. Econometric methods that focus on *summarizing* observed behavior without attempting to provide a deeper explanation for that behavior are referred to as reduced-form estimation (RFE). In the RL/ML literature it is referred to as "behavioral cloning" (BC) "imitation learning" (IM), or "apprenticeship learning" (AL). The analysis of identification treats π as known because with infinite data we could learn it perfectly. The preceding section showed how to derive π from the structure $\{\beta, r, p\}$, a process that we can summarize by the mapping $\pi = f(\beta, r, p)$. The identification problem concerns the invertibility of this mapping: given π is there a unique underlying structure $\{\beta, r, p\}$ that implies it? Unfortunately, without further restrictions the answer is negative, as shown by Hotz and Miller (1993), Rust (1994) and Magnac and Thesmar (2002). To understand why, consider the first step of the inversion process, uncovering Q from π. Using the

[24] This inversion process requires a combination of identifying assumptions to ensure a unique solution. The foundational works in both single-agent DDC and dynamic games exemplify these strategies. *Single-Agent DDC*: Rust (1987) imposes parametric functional forms (e.g., linear cost in mileage), assumes rational expectations so that transition probabilities may be treated as known, and fixes the discount factor (β). To resolve the observational equivalence wherein only cost differences are identified, local normalization is employed. Rust (1987) notes that because "the absolute level of c [cost] is not identified," he must "normalize by setting $c(0, \theta_1) = 0$" (p. 1015), thereby anchoring cost at zero mileage. *Dynamic Games*: Anderson et al. (2025) assume the reward function (r) is known (normalized to 1 for a win, 0 for a loss), impose stationarity on point outcome probabilities, employ a parametric logit specification for subjective beliefs, and introduce stochastic trembles (IID Type 1 extreme value shocks) to rationalize observed mixed strategies. In both cases, imposing restrictions on certain model primitives is necessary to identify others, rendering an otherwise intractable inverse problem solvable.

logit form of π in (18) we have

$$\log\big(\pi(a|s)/\pi(a'|s)\big) = Q(s,a) - Q(s,a'), \quad s \in S,\ a, a' \in A. \tag{30}$$

We already see that information on π is insufficient to recover all the Q values: only the *differences* in Q are identified but not all $|S||A|$ possible values of $Q(s,a)$. Indeed, the structure $\{\beta, r, p\}$ contains $1 + |S|(|A|-1) + |S|^2|A|$ parameters which is more than then $|S|(|A|-1)$ parameters of π, so we will generally have "more equations than unknowns" and the underlying structure will only be partially identified.

Identification requires additional restrictions on $\{\beta, r, p\}$. Empirical studies commonly impose the assumption that r and p have known parametric functional forms that depend on a relatively small number of parameters θ. In some situations the reward function can be treated as known, and this facilitates the identification of subjective beliefs $p(s'|s,a)$. For example Anderson et al. (2025) are able to identify the subjective beliefs of professional tennis servers about how their serve strategies affects their probability of winning a tennis game. These beliefs take the form of a transition $p(s'|s,a)$ where s denotes the point state of the game and a denotes serve direction. Identification of beliefs is possible because it is reasonable to assume r is known: i.e. the server earns a reward of 1 if they win the game and 0 otherwise, and due to reasonable parametric restrictions on p.

In many applications researchers are willing to assume agents have rational expectations which implies that their subjective beliefs about state transitions $p(s'|s,a)$ correspond to the actual probability governing these transitions and so p can be estimated non-parametrically given sufficient data, and thus can be treated as known but with restrictions on r this assumption is testable.[25] Many studies typically assume that the discount factor β is also known, however under the assumption of rational expectations and restrictions on rewards, β can be identified, see Abbring and Daljord (2020).

Following the DDC approach of local normalization, and assuming $\{\beta, p\}$ are known, identification of r depends on the assumption that either it has a known parametric functional form r_θ where the dimension of θ is less than $|S|(|A|-1)$, or that for each $s \in S$ there is a normalizing or anchor action a_s such that $r(s, a_s)$ is known. The latter assumption amounts to $|S|$ restrictions that imply there are only $|S|(|A|-1)$ unknown rewards to be recovered, the same number of free values of π. Hotz and Miller (1993) and Kang et al. (2025) show that in this "exactly identified" case, we can calculate $Q(s, a_s)$ from knowledge of $r(s, a_s)$ and $\pi(a_s|s)$ using equation (20) and then (30) allows us to recover all remaining Q values. Using the Q values we can back out the remaining unknown rewards r from the smooth Bellman equation (15).

Note that the assumption that we know $r(s, a_s)$ for some anchor action a_s in each state $s \in S$ is far from innocuous. If this assumption is wrong, even though the DDC model has enough

[25] Anderson et al. (2025) strongly reject the hypothesis the many professional tennis servers have rational beliefs about their own ability and those of their opponent, as embodied by $p(s'|s,a)$.

parameters to perfectly fit π, the estimated rewards will not equal the true rewards. In this case, counterfactual behavior predicted by the model with an incorrectly identified reward function can differ from what would actually happen under the true reward. Another way to say this is that in the absence of any prior restrictions on rewards, we can only identify a set of "observationally equivalent" rewards of the form $r(s,a) + h(s) - \beta \sum_{s'} h(s')p(s'|s,a)$ for any function $h : S \to R$. The set of all of these reward functions is called the partially identified set, and it defines an inherent limit on what can be learned about agents' rewards and emphasizes the importance of having accurate prior knowledge of at least some aspects of the reward function to obtain credible counterfactual predictions and welfare impacts of changes in policy and the environment. This class of reward transformations is precisely what the IRL literature, following the seminal work of Ng et al. (1999), calls potential-based reward shaping. In this framework, the function $h(s)$ is referred to as a 'potential function,' often denoted $\Phi(s)$. Any reward function $r'(s,a)$ created by adding the difference in potential $(\beta\Phi(s') - \Phi(s))$ to the original reward $r(s,a)$ will yield the exact same optimal policy π^*. This property, where multiple reward functions map to the same optimal behavior, is known as identification 'up to policy invariance.' For many applications in robotics and AI, such as imitation learning, achieving policy invariance is the end goal; the objective is to learn a reward function that can replicate an expert's behavior, not necessarily to uncover the expert's true, underlying reward. However, for the goals of structural econometrics—evaluating welfare, testing economic theories, and predicting behavioral responses to novel policy interventions (counterfactuals)—this ambiguity is a fundamental problem that must be resolved. Credible welfare analysis requires the uniquely identified reward function, not just one of many that happens to rationalize the observed data.

To resolve the identification ambiguity, the DDC and IRL literatures have developed distinct sets of identifying assumptions, each reflecting the typical problems and goals of the respective fields. These can be broadly characterized as a local, action-based normalization common in DDC versus a global, state-space connectivity condition used in IRL. Influential methods in the IRL literature, particularly Adversarial IRL (AIRL) (Fu et al., 2018a), propose a global condition on the transition dynamics known as 'reward-decomposability.' Intuitively, this assumption requires that all states in the MDP are 'linked'—meaning it is possible to eventually transit from any state to any other state through some sequence of actions. When this condition holds, observing choices across the entire state space creates a single, interconnected system of reward constraints. This allows the reward function (often assumed to be state-dependent, $r(s)$) to be identified globally up to a single additive constant, without needing to normalize a specific action in every single state. More recent work by Cao et al. (2021) extends these identifiability results to stochastic environments and provides conditions under which rewards can be recovered up to potential shaping transformations. However, a critical distinction for economic applications is that this powerful identification result for AIRL was originally proven under the restrictive assumption of deterministic state transitions, where a given state-action pair (s,a) leads to a single, known next state s'. This

assumption is often untenable in economic and social settings, which are inherently stochastic. For example, in consumer choice, a firm's marketing action (a) in a given market state (s) does not lead to a deterministic sales outcome (s'); it leads to a distribution over possible outcomes. Similarly, a patient's adherence to a treatment plan does not guarantee a specific health trajectory. While subsequent research has extended the analysis to stochastic environments, identification often requires imposing strong technical conditions, such as a full-rank requirement on the transition probability matrix (Cao et al., 2021). This essentially requires that the available actions from a state are powerful enough to generate linearly independent probability distributions over next states. This contrasts sharply with the DDC framework, which is designed from the ground up to accommodate stochasticity as a fundamental feature of the decision-making environment.

In contrast, the structural econometrics literature typically achieves identification through local normalizations, as we detail next.

Kalouptsidi et al. (2021) provides examples of certain counterfactuals that are identified even when r is only partially identified. They conclude that their "results call for caution while leaving room for optimism: although counterfactual behavior and welfare can be sensitive to identifying restrictions imposed on the model, there exists important classes of counterfactuals that are robust to such restrictions." (p. 385).

4 Inverse Reinforcement Learning

The IRL literature was initiated by Ng and Russell (2000) who identified two motivations for inferring the underlying rewards that are presumed to generate observed behavior: 1) "the potential use of reinforcement learning and related methods as potential models for human and animal learning" and 2) to provide a reward function that RL can use to construct an "intelligent agent that can behave successfully in a particular domain." Though they note that imitation and apprenticeship learning (IL and AL) can be used to learn a policy π, "the reward function often provides a much more parsimonius description of behaviour" so that IRL can constitute an effective form of AL. IRL has become an important method in many areas of robotics including development of autonomous drones and vehicles (AV) where there is insufficient data on decisions to human drivers under multitudes of different road conditions to develop effective AVs. Wang et al. (2025) note that IRL has "found great success in predicting trajectories through inferring cost or reward functions from expert demonstrations, and then using these functions to guide the behavior of self-driving vehicles in unseen driving environments."

This section provides a selective review of the IRL literature focusing on methods that overlap most closely with the structural DDC literature reviewed in section 3. We refer readers to surveys by Arora and Doshi (2021), Gleave and Toyer (2022), and Adams et al. (2022) for a broader and more in depth view of IRL. The latter survey provides further examples of how IRL uses "an expert to demonstrate the successful task and then a highly parameterized reward function could be transferred to the robot for optimization" and how IRL can generate more effective policies than

AL, particularly for new, counterfactual environments where "IRL has the added advantage that the reward function can be transferred to new environments with different dynamics.".

There are four broad classes of IRL algorithms: 1) max-margin, 2) Bayesian methods, 3) maximum entropy methods, and 4) adversarial methods. Ng and Russell (2000) introduced three different types of max-margin estimators, which search for a reward function for which payoffs under the observed policy are higher than any other policy (i.e. behavior), an approach that shares similarities with the "moment inequality" simulation estimator proposed by Bajari et al. (2007) discussed in section 3. Ng and Russell (2000) recognized the identification problem discussed in the previous section, namely there may be many reward functions that satisfy the max-margin inequalities (payoff inequalities) including the trivial solution $r = 0$.[26]

Bayesian IRL (BIRL) was introduced by Ramachandran and Amir (2007). They assumed π is a mixed strategy with the softmax form (18) that depends on Q functions which are implicit functions of r per the Bellman equations (6) and (7) and interpret the σ parameter in the softmax function for π as a rationality coefficient that governs how sharply the policy concentrates on high-value actions.[27] This setup allowed them to formulate a likelihood for the sequence of observed actions by the DM that depends on r. Using Bayes' Rule, they computed a posterior distribution for r, treating it as the unknown parameter with a prior distribution. Since it is infeasible to directly compute the posterior, BIRL uses MCMC to generate draws from the posterior distribution, with policy iteration used to solve for Q for each draw of r. This makes BIRL method computationally intensive for large state spaces similar to NFXP. Overall BIRL is similar to the Bayesian MCMC methods discussed in section 3, except that those methods use stochastic updates of Q similar to those produced by Q-learning to gradually compute Q over the course of the MCMC simulations rather than use policy iteration to find Q for each MCMC draw of r.

A seminal contribution to the IRL literature is Ziebart et al. (2008), who introduced *maximum entropy IRL* as a means of justifying the use of the softmax/logit probability for $\pi(a|s)$ as the probability with maximum entropy subject to certain constraints that make expectations of certain functions known as *features* equal the sample averages of these features.[28] They proposed to estimate the parameters of a reward function that is a linear combination of a set of known features using a partial likelihood for observed (a, s) pairs that is identical to the partial likelihood (26) used in the DDC literature. They applied this method to model the route choices of 25 Yellow cab drivers in Pittsburgh with a reward function that involves four features: road type, speed, lanes and transitions. They showed that the estimated model could correctly predict nearly 80% of the cab

[26]Building on Ng et al. (1999), who showed the optimal policy is invariant under reward transformations of the form $r'(s, a) = \alpha r(s, a) + \beta + \gamma\Phi(s') - \Phi(s)$ for $\alpha > 0$ and potential function Φ, Ng and Russell (2000) termed this identification "up to policy invariance." This suffices for imitation learning but not for DDC, which requires unique identification to enable hypothesis testing and welfare analysis.

[27]Small σ makes the policy concentrate almost entirely on the highest-value action, while large σ spreads probability nearly uniformly across actions. Because the softmax formula depends only on Q/σ, the reward scale and σ cannot both be identified, so DDC applications typically fix $\sigma = 1$ (e.g. Adusumilli and Eckhardt (2025)). When calibration data are available, Ghosal et al. (2023) show how to estimate β (equivalently $\sigma = 1/\beta$) instead of fixing it.

[28]Ziebart et al. (2010) extended these ideas to MDPs and called it *maximum causal entropy IRL*.

paths in a holdout or evaluation sample and that their new method was a significant improvement over other IRL methods such as max-margin.

A fourth class of IRL methods uses adversarial training, building on Generative Adversarial Imitation Learning (GAIL). Fu et al. (2018b) introduced Adversarial IRL (AIRL), which extends GAIL by learning a *disentangled* reward function that is robust to changes in environment dynamics. AIRL formulates IRL as a two-player game between a policy generator and a discriminator. The key innovation is structuring the discriminator as $f(s, a, s') = g_\theta(s, a) + \gamma h_\phi(s') - h_\phi(s)$, where $g_\theta(s, a)$ represents the learned reward function and h_ϕ is a state-dependent *shaping function*. This discriminator structure is a practical implementation of the potential-based reward shaping discussed in Section 3.3, where $h_\phi(s)$ plays the role of the potential function. As noted in our earlier discussion, this approach allows AIRL to achieve identification via the global 'reward-decomposability' assumption, but its foundational proof relies on deterministic dynamics, a key point of departure from the stochastic environments central to DDC modeling. This decomposition disentangles the reward from dynamics-specific features, enabling learned rewards to transfer across environments with different transition probabilities $p(s'|s, a)$. The discriminator is trained to distinguish expert trajectories from policy-generated trajectories, while the policy maximizes the learned reward. AIRL achieves identification of the reward function "up to policy invariance" (i.e., up to potential-based shaping) without requiring explicit solution of the Bellman equation during training.[29]

These ideas evolved into the already cited work of Haarnoja et al. (2017) who characterized π as the policy that maximizes the expected discounted entropy-regularized payoff given in equation (9). Various criteria are used to estimate r such as "occupancy matching" (Ho and Ermon (2016), Yue et al. (2023)). For our purposes we focus on IRL where the likelihood function is the estimation criterion, such as Ermon et al. (2015), Zeng et al. (2023), Zeng et al. (2024), Barzegary and Yoganarasimhan (2022) and Kang et al. (2025).[30] This results in a nearly complete equivalence between the DDC and IRL literatures, with identical softmax/logit formulas for the policy $\pi(a|s)$ (18) that depend on the choice-specific or "soft Q" values that satisfy the "soft Bellman equation" in equation (15). Overall, DDC and maximum entropy IRL result in mathematically identical optimization problems for recovering r from data on states and actions of a sample of DMs. The main difference is in the algorithms used to solve the DM's DP problem.

In DDC the Q functions are computed by non-stochastic algorithms such as SA or PI, or in large state spaces by parametrizing Q or the value function V and finding parameters ϕ that minimize the squared "Bellman residuals" as in the SEES estimator, (29). All of these methods can be described as *model-based* because they require knowledge of the transition probability $p(s'|s, a)$, or

[29] However, Theorem C.1 in Fu et al. (2018b) requires deterministic dynamics: "we also assume we have deterministic dynamics." Since economic applications typically involve stochastic transitions $p(s'|s, a)$, this assumption limits AIRL's applicability to most DDC problems. Fu et al. (2018b) demonstrate that AIRL achieves comparable performance to GAIL on standard imitation benchmarks but significantly outperforms GAIL on transfer tasks requiring generalization across different environment dynamics.

[30] Zeng et al. (2023) and Zeng et al. (2024) maximize a discounted log-likelihood function.

at least p must be estimable from data. Adams et al. (2022) notes that most IRL methods are model-based. However, we will focus on model-free IRL that does not require specification of a function form for p or even require that it be explicitly estimated non-parametrically. Instead, new IRL methods inspired by model-free approaches such as Q-learning are able to approximate the Q functions using only realizations from p without knowing p itself. But how is it possible to generate realizations s' from $p(s'|s,a)$ without knowing p itself?

Model-free IRL solves this problem in a clever way: it uses already realized transitions in demonstration datasets to provide the "simulator" needed by RL algorithms to train the Q and values functions. The insight is that the data on observed states and decisions provides the "environment" and the state transitions needed by RL to find the value and Q functions needed to recover r, reminiscent of the idea of *experience replay* in the deep Q learning literature, Mnih et al. (2015). To get a better understanding of these methods, we will provide a more in-depth discussion of two recent contributions the IRL literature that can recover r from data in a model-free manner. One can be regarded as an IRL version of the CCP estimator and the other is similar to the SEES estimator (29). Both of these methods use parametric function approximation methods (expressing V and Q as outputs of neural networks or linear combinations of a set of basis functions) that enables them to handle problems with large state spaces.

The RL and SE/DDC literatures use "model-free" and "model-based" differently when analyzing learning from historical data.[31]

Adusumilli and Eckhardt (2025) (AE) introduce a model-free two step CCP estimator similar to the one described in the previous section but using RL-inspired methods to avoid estimating or making parametric assumptions about $p(s'|s,a)$. They do assume that the reward function is a linear combination of known "features", i.e. $r_\theta(s,a) = \sum_{k=1}^{K} \theta_k r_k(s,a)$. Their estimator is inspired by the temporal difference (TD) method of RL, and they apply it to approximate the functions R and Q_ϵ (defined in equations (22) and (24), respectively) that enter as covariates in the CCP π in equation (25), using transitions in the data to estimate these functions without having to explicitly estimate $p(s'|s,a)$. To accommodate empirical applications with large state spaces they propose parameterizing R and Q_ϵ as linear combinations of a set of known basis functions that depend on (s,a). Let $\nu(s,a) = \{\nu_1(s,a), \ldots, \nu_J(s,a)\}$ denote J basis functions that are used to approximate R and Q_ε by a regression that uses the basis functions evaluated at values of (s,a) in the data set as regressors. We will describe AE's approach to approximating R, since the same procedure is used to approximate Q_ε and is omitted for brevity. Let R_ϕ denote an approximation to R based on a $J \times 1$ parameter vector ϕ of the form $R_\phi(s,a) = \nu(s,a)\phi = \sum_{j=1}^{J} \theta_j \nu_j(s,a)$. AE propose to to

[31] We thank an anonymous reviewer for highlighting this important terminological distinction. In RL, an algorithm is *model-free* if it uses raw transition tuples (s,a,s') for updates (e.g., TD learning) without explicitly estimating or representing the transition probability $p(s'|s,a)$, whereas it is *model-based* if it explicitly constructs or estimates p for planning (see Chapter 8 of Sutton and Barto (2020)). In SE/DDC, any method relying on historical data to infer structure is typically considered "model-based" since, as the reviewer noted, "one can estimate p from the triplets $[(s,a,s')]$ and generate the triplets from p," making the distinction "quite thin" when both approaches rely on the same historical transitions. Throughout this article, we follow the RL terminology when describing IRL methods.

choose $\hat{\phi}$ to minimize the mean-squared TD error given by

$$\hat{\phi} = \underset{\phi}{argmin}\, E\left\{[\vec{r}(s,a) + \beta R_\phi(s',a') - R_\phi(s,a)]^2\right\}, \tag{31}$$

where E denotes the expectation operator over (s, a, s', a'). Using results from Tsitsiklis and Roy (1997) on the convergence of TD learning with linear function approximation, AE show that $\hat{\phi}$ can be computed using the sample expectation operator E_N in (31) using the formula

$$\hat{\phi} = \left[E_N\{\nu(s,a)[\nu(s,a) - \beta\nu(s',a')]^T\}\right]^{-1} E_N\{\nu(s,a)\vec{r}(s,a)\}, \tag{32}$$

and the implied estimate $\hat{R}(s,a) = \phi(s,a)\hat{\phi}$ of the features or "covariates" for the θ in (25). We also have to approximate the second "correction term" $\hat{Q}_\varepsilon(s,a)$, which can be estimated via regression similar to (32) but it requires a first stage non-parametric estimate of $\pi(a|s)$ to construct estimates of the expected choice-specific unobserved shocks $E\{\varepsilon(a)|s,a\} = -\log(\pi(a|s))$.[32] Using the estimated $\hat{R}(s,a)$ and $\hat{Q}_\varepsilon(s,a)$ values, they can compute a second stage pseudo-maximum likelihood (PMLE) estimator of θ by maximizing the partial likelihood (26) using the formula for $\pi_\theta(a|s)$ in (25). Consistency of this first stage estimator requires specification of a sieve with J basis functions $\{\nu_1, \ldots, \nu_J\}$ that increases with sample size N at an appropriate rate and with sufficient linear independence to allow any function R to be perfectly approximated by its projection $\Pi_\nu R$ in the limit as $J \to \infty$. The PMLE is affected by "estimation noise" in the $\hat{R}$ and $\hat{Q}_\varepsilon$ functions that can introduce bias and noise. To deal with this, AE propose a third estimation stage that uses the PMLE to create a modified score (i.e. gradient) of the partial log-likelihood (26) to create a robust (Neyman orthogonal) version of the score function which they used to their preferred estimator $\hat{\theta}$ as the value that sets the modified score to zero following the approach of Chernozhukov et al. (2022).

AE proposed a different estimator they call AVI (for Approximate Value Iteration) that allows for nonlinear parametrizations of the R and Q_ε functions which are computed using an iterative procedure for updating the parameter $\hat{\phi}_k$ is the estimated parameters determining $\hat{R}_k(s,a) = R_{\hat{\phi}_k}(s,a)$ as follows

$$\hat{\phi}_{k+1} = \underset{\phi}{argmin}\, E_N\left\{\left[\vec{r}(s,a) + \beta R_{\hat{\phi}_k}(s',a') - R_\phi(s,a)\right]^2\right\}. \tag{33}$$

These iterations continue until the sequence $\{\hat{\phi}_k\}$ converges and the resulting estimator of R is $\hat{R} = R_{\hat{\phi}_k}$ where $\hat{\phi}_k$ is the last value of the iterative procedure (33), and similarly for $\hat{Q}_\varepsilon$.[33] AE

[32] AE normalize σ to 1 since it cannot be separately identified from the θ parameters.

[33] It is possible to extend their approach to reward functions $r_\theta(s,a)$ where θ does not enter as a linear combination of a set of known features $\vec{r}(s,a)$. However in that case, we can no longer express $Q_r(s,a) = R(s,a) * \theta$, and in general θ will enter Q_r in a non-linear fashion. The TD estimator can still be adapted to work in this situation except that now there must be a nested TD subroutine that recomputes $\hat{Q}_r$ as a function of θ each time θ is updated in an outer optimization algorithm that searches over θ to maximize the partial likelihood function (26).

conclude that "A range of Monte Carlo simulations using a dynamic firm entry problem, a dynamic firm entry game and two versions of the famous Rust (1987) engine replacement problem show that the proposed algorithms work well in practice."[34]

Kang et al. (2025) developed an estimator they call GLADIUS (Gradient-based Learning with Ascent-Descent for Inverse Utility learning from Samples) that is 1) model-free, 2) does not require a first stage non-parametric estimation of $\pi(a|s)$, and 3) does not require a parametric specification for rewards. Instead, GLADIUS is an iterative algorithm that uses gradient ascent/descent to maximize a penalized likelihood function similar to the SEES estimator (29). However instead of parametrizing r as a function of "structural parameters" θ and using using auxilliary parameters ϕ to approximate the Q functions, GLADIUS relies on flexible parametrization of the Q function as $Q_{\theta_1}(s,a)$ and the conditional expectation of the value function $E\{V(s')|s,a\}$ as $EV_{\theta_2}(s,a)$ for parameters θ_1 and θ_2 of flexible approximations to Q and EV such as DNNs. Once GLADIUS converges, returning parameters $\hat{\theta}_1$ and $\hat{\theta}_2$, rewards are estimated as the function

$$r(s,a) = Q_{\hat{\theta}_1}(s,a) - \beta EV_{\hat{\theta}_2}(s,a), \tag{34}$$

i.e. they are "backed out" from the fixed point equation for smoothed Q function, (15).

Both GLADIUS and SEES use an objective function that maximizes a penalized likelihood where the penalty term is a "mean squared Bellman error" for the Q functions. That is, if we express Q as a fixed point $Q = \Lambda_\sigma(Q)$, Q minimizes the quantity $\rho_{BE}(Q) = E\{[Q(s,a) - \Lambda_\sigma(Q)(s,a)]^2\}$ over a joint distribution of (s,a) pairs. SEES is a model-based approach that uses an assumed parametric functional form for rewards, $r_\theta(s,a)$ and knowledge (or estimated version thereof) of the transition probability' $p(s'|s,a)$ to compute the Bellman operator $\Lambda_{\theta,p,\sigma}$, where we subscript Λ_σ to emphasize its dependence on θ as well as p. Thus, SEES calculates the mean squared Bellman error $\rho_{BE}(Q)$ numerically using deterministic methods such as numerical quadrature to compute the conditional expectations of the Q functions.

GLADIUS is a model-free approach inspired by Q-learning that uses transitions (s',s,a) in the data to construct an estimate of the mean squared Bellman error $\rho_{BE}(Q)$. That is, GLADIUS seeks to approximate the quantity

$$\Lambda_\sigma(Q)(s,a) = r(s,a) + \beta \sum_{s'} V(s')p(s'|a,d) \tag{35}$$

by averaging realizations of the form

$$\tilde{\Lambda}_\sigma(Q)(s',s,a) = r(s,a) + \beta V(s'), \tag{36}$$

[34]See Khwaja and Srivastava (2025) for an alternative approach to AE's TD methods that they call RLTD-CCS that combines the TD algorithm from RL with the Conditional Choice Simulator (CCS) estimator of Hotz et al. (1994) that relies on forward simulations of paths to produce a Monte Carlo estimate the $Q_\theta(s,a)$ and hence is a model-based approach to estimation.

using observed (s, a, s') transitions in the sample. It follows that $E\{\tilde{\Lambda}_\sigma(s', s, a)|s, a\} = \Lambda_\sigma(Q)(s, a)$ so we can use transitions (s', s, a) in the data to calculate $\Lambda_\sigma(Q)(s', s, a)$ values, and then use these as dependent variables in a regression to estimate its conditional expectation $\Lambda_\sigma(s, a)$. This suggests a strategy of estimating Q by searching for θ_1 to minimize the mean squared temporal difference error $\rho_{TD}(Q)$ given by

$$\rho_{TD}(Q) = E\{[\Lambda_\sigma(Q)(s', s, a) - Q(s, a)]^2\}, \tag{37}$$

where, using the terminology of the RL literature, $\tilde{\Lambda}_\sigma(Q)(s', s, a) - Q(s, a)$ is the TD error.

However Kang et al. (2025) note an important complication: the true fixed point Q will minimize the mean squared Bellman error $\rho_{BE}(Q)$ but it will not necessarily minimize the mean squared TD error. To see, this, the note the following identity

$$\rho_{TD}(Q) = \rho_{BE}(Q) + \beta^2 E\{[V(s') - EV(s, a)]^2\}, \tag{38}$$

where the second term reflects expected "heteroscedasticity" in the TD error, i.e. it is the average of the conditional variances given by $H(V)(s, a) = E\{[V(s') - EV(s, a)]^2|s, a\}$. Since V is a function of Q from (16), we write V as $V(Q)$ and the heteroscedasticity term as $H(Q)$. It it follows from (38) that the Q that minimizes $\rho_{TD}(Q)$ will not necessarily be the same as the Q that minimizes $\rho_{BE}(Q)$. To deal with this issue, they express the mean squared Bellman error as $\rho_{BE}(Q) = \rho_{TD}(Q) - H(Q)$. Though $H(Q)$ is also an unknown quantity Kang *et. al.* note that it can be approximated as the solution to

$$H(Q)(s, a) = \min_{v \in R} \sum_{s'} [V(Q)(s') - v]^2 p(s'|s, a). \tag{39}$$

Of course the v that minimizes (39) is $EV(s, a)$. Kang et al. (2025) parametrize Q and EV by DNNs as Q_{θ_1} and EV_{θ_2}, respectively, to obtain an estimation criterion that is a penalized likelihood function similar to SEES, except it is a max-min problem given by

$$(\hat{\theta}_1, \hat{\theta}_2) = \underset{\theta_1}{argmax}\ \underset{\theta_2}{argmin} \log(L(\theta_1)) - \lambda\rho(\theta_1, \theta_2), \tag{40}$$

where λ is a penalization parameter, and $\rho(\theta_1, \theta_2)$ is the estimate of the mean squared Bellman error given by

$$\rho(\theta_1, \theta_2) = \frac{1}{N} \sum_i^N [\Lambda_\sigma(Q_{\theta_1}(s'_i, s_i, a_i) - Q_{\theta_1}(s_i, a_i)]^2 - \beta^2 [V(Q_{\theta_1})(s'_i) - EV_{\theta_2}(s_i, a_i)]^2 \tag{41}$$

Computation of the penalty term can be further reduced to limiting the summation to a subset of states s_i where the corresponding action a_i equals the *normalizing action.* The reason is that Kang et al. (2025) impose the identifying assumption that for each s there is an action a_s called the normalizing action for which $r(s, a_s)$ is known. From our discussion of identification in section

3.2, this restriction on rewards implies that we can determine all Q values by enforcing the fixed point constraint $Q = \Lambda_\sigma(Q)$ only for the subset of pair (s, a_s) which enables us to determine the corresponding Q at the normalizing action, $Q(s, a_s)$ for all $s \in S$. Then given $Q(s, a_s)$ the remaining $Q(s, a)$ values are determined from $\pi(a|s)$ using the Hotz-Miller inversion formula (30).

Kang et al. (2025) establish an important result on the global convergence of GLADIUS. First, under a "realizability assumption" that there exists θ_1^* such that $Q^* = Q^*_{\theta_1^*}$, where Q^* is the true Q function (as well as some other regularity conditions), they show that if $\hat{Q}_{T,N}$ is the estimate of Q^* after T iterations of the GLADIUS algorithm using a data set with N observations on agent's states and transitions (s', s, a), then

$$E_N \left\{ [Q^*(s, a) - \hat{Q}_{T,N}(s, a)]^2 \right\} \leq O(1/T) + O(1/N). \tag{42}$$

They note that "To the best of our knowledge, no prior work has proposed an algorithm that guarantees global optimum convergence of the minimization problem that involves the [mean squared Bellman residual term]" (see Section 2 for details on this terminology distinction). They compare GLADIUS to several other DDC estimation methods using the Rust (1987) bus engine problem and a high-dimensional version of this problem. In the high-dimensional version, they added between 2 and 100 additional irrelevant state variables (i.e., variables that have no effect on cost of maintaining the bus or its mileage transitions, and thus on the Q values and decision rule π), each taking on 20 possible values, thus massively increasing the size of the state space. They conclude that "We find that, on average, our method performs quite well in recovering rewards in both low and high-dimensional settings. Further, it has better/on-par performance compared to other benchmark algorithms in this area (including algorithms that assume the parametric form of the reward function and knowledge of state transition probabilities) and is able to recover rewards even in settings where other algorithms are not viable."[35]

We have provided a very selective survey of the IRL literature, focusing on two recent contributions that are closely related to DDC and offer promising new methods for structural estimation. We have emphasized the distinction between model-based and model-free estimation methods, but we wish to make clear that not all IRL methods are model-free. For example Zeng et al. (2024) use non-parametric methods to estimates p to provide a "generative world model" in a model-based version of IRL and "Through extensive experiments, we demonstrate that our algorithm outperforms existing benchmarks for offline IRL and Imitation Learning, especially on high-dimensional robotics control tasks." Thus, it is not obvious that model-free approaches are necessarily superior to model-based approaches, though we agree that good models of p should be used, since "inaccurate models of the world obtained from finite data with limited coverage could compound inaccuracy in estimated rewards." It is also clear that if we are interested in doing a counterfactual

[35] Kang et al. (2025) also note GLADIUS corrects a problem in IRL methods that use "occupancy matching" as the estimation criterion such as Ho and Ermon (2016), Garg et al. (2022), and Yue et al. (2023). They show that occupancy matching is not guaranteed to consistency estimate the true Q function, and "This implies that r cannot be inferred from Q using the Bellman equation after deriving Q using occupancy matching."

that involves changing p, model-based methods will probably be necessary to estimate/construct this new p and then train the model to calculate the new optimal decision rule $\pi(a|s)$ corresponding to counterfactual p.

Many IRL variants that appear distinct prove closely related upon examination. Ziebart et al. (2010)'s maximum causal entropy IRL, for instance, is nearly isomorphic to DDC models under the AS/CI/EV assumptions discussed in section 3. As Gleave and Toyer (2022) notes: "algorithms based on MCE IRL have scaled to high-dimensional environments. Maximum Entropy Deep IRL Wulfmeier et al. (2016) was one of the first extensions, and is able to learn rewards in gridworlds from pixel observations. More recently, Guided Cost Learning Abbeel (2016) and AIRL have scaled to continuous control tasks." Large-scale applications include Barnes et al. (2024) using IRL to infer route preferences from Google Maps data and Zhao and Liang (2023) applying AIRL to Shanghai taxi trajectories.[36]

5 Reinforcement Learning from Human Feedback

The structural estimation and IRL literatures are primarily concerned with recovering an agent's underlying reward function from observations of their enacted, often optimal, behavior. A distinct but related paradigm, Reinforcement Learning from Human Feedback (RLHF), has recently emerged to address settings where expert demonstrations are either unavailable, prohibitively expensive to collect, or may not represent the true desired behavior. Instead of inverting an observed policy π to find the reward function that rationalizes it, RLHF learns a reward function directly by eliciting preferences from a human evaluator over pairs of machine-generated behaviors. While this approach has become the predominant method for aligning large language models (Ouyang et al., 2022), its methodology is general and was first demonstrated as a viable technique for training agents in standard reinforcement learning domains, such as simulated robotics and video games, without access to an explicit, pre-defined reward function (Christiano et al., 2017).

The canonical RLHF framework is best understood as a three-step process that combines supervised learning, preference modeling, and policy optimization. The objective is to fine-tune a base policy, typically a large pretrained model, to align its behavior with human judgments. The first step, supervised fine-tuning (SFT), initializes the policy in a sensible region of the parameter space. A small, high-quality dataset of human-written demonstrations of the desired task is collected, and a pretrained policy is fine-tuned via standard maximum likelihood to produce an initial policy, which we denote π^{SFT}.

The second and central step is the learning of a reward model, $r_\theta(s, y)$, parameterized by θ. Note that unlike the per-action rewards $r(s, a)$ in the preceding MDP framework, the reward here is assigned to a complete trajectory or output y generated from state s. This step directly applies the principles of discrete choice modeling discussed in Section 3. A dataset $\mathcal{D}$ of human preferences

[36]The Shanghai application succeeded because road networks provide deterministic transitions and the goal was matching observed trajectories rather than counterfactual analysis.

is collected, consisting of tuples (s, y_w, y_l), where s is a context or state, and y_w and y_l are two trajectories or outputs generated by the policy, with y_w being the "winner" preferred by the human evaluator and y_l the "loser." The reward model r_θ is then trained to predict these choices. Assuming the human preferences can be described by a random utility model, where the latent utility of an output y in context s is given by $r_\theta(s, y)$, the probability that y_w is preferred over y_l can be modeled using a logit or softmax function. The reward model parameters θ are estimated by minimizing the negative log-likelihood of the human choices, which corresponds to a binary cross-entropy loss:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(r_\theta(s, y_w) - r_\theta(s, y_l)\right)\right], \tag{43}$$

where $\sigma(\cdot)$ is the logistic sigmoid function. The learned r_θ thus serves as a scalar proxy for human preferences.

In the third and final step, the policy is optimized using reinforcement learning, with the learned reward model r_θ providing the reward signal. The policy, initialized from π^{SFT}, is updated to maximize the expected reward given by r_θ. A critical component of this optimization, first introduced in this context by Ziegler et al. (2019), is the inclusion of a regularization term that penalizes the divergence of the learned policy, π_ϕ, from the initial supervised policy, π^{SFT}. The policy is then optimized to maximize the expected reward from the human-preference model, regularized by a KL-divergence penalty against the initial policy π^{SFT}. The objective function for the policy optimization is:

$$J(\phi) = \mathbb{E}_{s\sim\mathcal{D}, y\sim\pi_\phi(\cdot|s)}[r_\theta(s, y)] - \beta_{KL}\mathbb{E}_{s\sim\mathcal{D}}[D_{KL}(\pi_\phi(\cdot|s) \,||\, \pi^{SFT}(\cdot|s))], \tag{44}$$

where D_{KL} is the Kullback-Leibler divergence and β_{KL} is a hyperparameter controlling the strength of the penalty. The KL-divergence term in this objective serves two essential functions: it acts as an entropy bonus to encourage exploration and, more importantly, it constrains the policy to remain close to the distribution of coherent outputs learned during the SFT phase. This prevents the policy from learning to generate adversarial or out-of-distribution outputs that "hack" the fixed reward model by exploiting its imperfections to achieve a high score with degenerate behavior.

While RLHF has proven exceptionally effective at aligning model behavior with human preferences (Stiennon et al., 2020), several open issues remain. The framework does not aim to solve the identification problem in the strict econometric sense; the learned reward model r_θ is a proxy for preference, not necessarily the uniquely identified "true" utility function required for welfare analysis. Furthermore, the notion of "human feedback" is a significant simplification, as the preferences being optimized are those of a small, non-representative group of paid labelers, raising important questions about whose values are being embedded in these systems. The practical implementation also presents a substantial operational challenge, as collecting a large, high-quality dataset of preferences requires a significant investment in both cost and meticulous quality control. Finally, the fine-tuning process can lead to a degradation of performance on standard academic

benchmarks, a phenomenon sometimes referred to as an "alignment tax." Mitigating this requires further methodological refinements, such as mixing pretraining gradient updates into the policy optimization process, as demonstrated by the PPO-ptx algorithm of Ouyang et al. (2022). Despite these challenges, RLHF represents a powerful and scalable approach for directing the behavior of complex generative agents in domains where rewards are difficult to specify but straightforward to evaluate through comparative judgment.

5.1 Recent Developments: Direct Policy Optimization and Self-Rewarding

Building upon the canonical RLHF framework, recent research has sought to simplify the multi-stage training pipeline and address the potential instabilities of reinforcement learning. A significant development in this direction is Direct Preference Optimization (DPO), a method introduced by Rafailov et al. (2023). DPO reframes the entire alignment problem as a direct supervised learning task, eliminating the need for an explicitly trained reward model and the complexities of RL-based policy optimization.

The key insight of DPO is the derivation of a closed-form relationship between the optimal policy of the KL-regularized reward maximization problem and the underlying reward function itself. For notational simplicity in the following derivation, we let $r(s, y)$ denote the ground-truth reward function that the trained model $r_\theta(s, y)$ aims to approximate. Recall that the theoretical optimal policy π_r for the objective in equation (44) can be expressed in terms of a reference policy π^{SFT} and the reward function $r(s, y)$ as:

$$\pi_r(y|s) = \frac{1}{Z(s)} \pi^{SFT}(y|s) \exp\left(\frac{1}{\beta_{KL}} r(s, y)\right), \tag{45}$$

where $Z(s)$ is a partition function that depends only on the state s. By rearranging this expression, the reward function can be reparameterized in terms of the optimal policy π_r and the reference policy π^{SFT}:

$$r(s, y) = \beta_{KL} \log\left(\frac{\pi_r(y|s)}{\pi^{SFT}(y|s)}\right) + \beta_{KL} \log(Z(s)). \tag{46}$$

Substituting this reparameterized reward into the preference model loss from equation (43) causes the state-dependent partition function term $\beta_{KL} \log(Z(s))$ to cancel out. This yields a loss function that depends only on the policy π being optimized and the fixed reference policy π^{SFT}. The DPO loss is thus a simple binary cross-entropy objective over policy likelihoods:

$$\mathcal{L}_{DPO}(\phi; \pi^{SFT}) = -\mathbb{E}_{(s, y_w, y_l)\sim\mathcal{D}} \left[\log \sigma\left(\beta_{KL} \log \frac{\pi_\phi(y_w|s)}{\pi^{SFT}(y_w|s)} - \beta_{KL} \log \frac{\pi_\phi(y_l|s)}{\pi^{SFT}(y_l|s)}\right)\right]. \tag{47}$$

This objective can be optimized directly on the policy parameters ϕ using standard supervised learning techniques with a static, offline preference dataset. The gradient of this loss intuitively increases the likelihood of preferred responses y_w while decreasing the likelihood of dispreferred

responses y_l, relative to the reference policy. By recasting the constrained optimization problem as a simple likelihood maximization, DPO offers a more stable, computationally efficient, and easier-to-implement alternative to the multi-stage RLHF pipeline.

A subsequent innovation, Self-Rewarding Language Models, leverages the DPO framework to create an iterative self-improvement loop that reduces the reliance on static, human-annotated data (Yuan et al., 2024). In this paradigm, a single language model acts as both the instruction-following agent and its own reward model. The process begins with a base model that is fine-tuned on a seed dataset to learn both instruction-following and evaluation capabilities (LLM-as-a-Judge). In each subsequent iteration, the current model generates a new preference dataset for itself by producing multiple responses to a set of prompts and then evaluating its own outputs to assign scores. These scores are used to construct preference pairs (y_w, y_l), which form an AI-generated feedback dataset. The model is then fine-tuned on this new dataset using the DPO loss from equation (47). This iterative process, where the model generates its own training data and provides its own reward signal, has been shown to create a "virtuous circle" of improvement, where enhancements in instruction-following ability lead to better reward modeling, which in turn fuels the next round of policy improvement. This approach demonstrates a path toward scaling alignment beyond the limits of human data collection by enabling models to autonomously refine their own capabilities.

6 Conclusion

We have compared structural DDC estimation, IRL, and RLHF; we now discuss key differences in their practical applications. An important difference in the IRL and DDC literatures is that the latter is more focused on statistical inference of the estimated reward function, deriving approximate distributions for sampling errors. As a result, many DDC applications involve estimation of smaller, more parsimonious models to gain new insights into human decision making and test behavioral hypotheses. The IRL literature is more focused on uncovering a reward function that can be used to train large scale applications using RL. IRL has been used for problems like route choice requiring estimation of thousands or millions of parameters. The large number of parameters and complexity of such reward functions makes it challenging to evaluate them from the perspective of statistical inference and insight into human behavior. However the "proof is in the pudding": robotic and AI applications achieve sophisticated behavior after RL training using IRL-learned reward functions (themselves learned from limited human demonstrations).

Both DDC and IRL literatures have both greatly benefitted from the use of Deep Neural Networks (DNNs) to approximate value functions in problems with large state spaces. Though it is still not formally established, the use of DNNs and other function approximation methods do seem to break Bellman's "curse of dimensionality" (at least in a practical sense) allowing us to formulate and estimate increasingly large and more realistic models.

A challenging area for further work is to relax and test the strong rationality assumptions

underlying DDC and IRL models. There are two key assumptions: 1) rational expectations (i.e. the subjective beliefs about the decision-dependent law of motion for state variables coincides with the true one), and 2) the ability to perfectly optimize given one's beliefs. If these assumptions do not hold, rewards inferred from human behavior will be distorted by our attempt to "rationalize" behavior that may not be fully rational. The notion that human behavior is more likely to be only "boundedly rational" goes back to the work of Simon (1957), and it is exemplified by the fact that chess algorithms trained using RL such as Alpha-Zero consistently beat the best human Grand Masters. RL was able to do this without any need for human data and IRL because 1) the reward function is known *a priori* (i.e. 1 if win, 0 if lose), and 2) the algorithm as trained via self-play.

The "model-free" approach to inferring rewards without explicitly specifying beliefs that is used in some recent work in IRL contrasts largely model-based estimation approach in the DDC literature. Though estimation using fewer assumptions is generally an attribute, model-free estimation does entail the strong implicit assumption that agents we are modeling have rational expectations. To the extent that humans are boundedly rational, we would like to understand whether suboptimal behavior is due to distorted or irrational beliefs or the inability to perfectly optimize. In simpler environments that are less strategically complex than chess, humans may be more likely to perfectly optimize but if their beliefs are not rational, the resulting behavior will still be suboptimal from the standpoint of a rational observer.

For example, Anderson et al. (2025) showed that elite tennis professionals use suboptimal serve strategies, and demonstrated how their behavior can be explained by distorted subjective beliefs about their own strengths and weaknesses as servers as well as their opponents. This opens a path for DDC and IRL models to improve human performance using data and model-free approaches that enables them to correct distorted subjective beliefs. However there are many high stake strategic situations (e.g. a wartime scenario) where the relevant data may be too scarce, leaving any decision maker no choice but to rely on an implicit "mental model" of the situation. In such situations neither DDC nor IRL methods may prove useful because of their dependence on good models of beliefs. More work needs to be done on how it might be possible to construct relatively accurate mental models (i.e. beliefs about how different actions affect outcomes), in unique situations where relevant data is extremely limited.

We conclude that the limitations confronting DDC and IRL are not computational but rather inherent to inferring rewards and beliefs from data on states and actions, as noted in our discussion of the identification problem in section 2.4. The solution to the identification problem requires us to impose strong prior restrictions (assumptions) about rewards and beliefs.

If these assumptions are wrong, counterfactual predictions from these models may not be accurate even though the models may provide a good fit to the demonstration data used to estimate them. However practical experience suggests that even if our assumptions may not be precisely correct and human decision makers may not be fully rational, the assumptions used to identify DDC and IRL models appear to be sufficiently good approximations to produce realistic counterfactual

simulations and leverage limited human demonstration data to train artificially intelligent agents to perform well in tasks previously requiring human expertise. The rapid growth in the DDC and IRL literatures suggests that despite the limitations and challenges we have noted, these methods have enormous practical value. We are likely to see continued rapid progress enabling these methods to be even more widely used to transform economics, finance and artificial intelligence.

References

Abbeel, Chelsea Finn Sergey Levine Pieter, "Guided Cost Learning: Deep Inverse Optimal Control via Policy Optimization," *arXiv*, 2016, *arXiv:1603.00448v3.*

Abbring, Jaap and Øystein Daljord, "Identifying the discount factor in dynamic discrete choice models," *Quantitative Economics*, 2020, *11* (2), 471–501.

Abdelnabi, Sahar, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd, "RLHF Deciphered: A Critical Analysis of Reinforcement Learning from Human Feedback for LLMs," *arXiv preprint arXiv:2404.08555*, 2024. Available at https://arxiv.org/abs/2404.08555.

Adams, Stephen, Tyler Cody, and Peter A. Beling, "A survey of inverse reinforcement learning," *Artificial Intelligence Review*, 2022, *55*, 4307–4436.

Adusumilli, Kaun and Dita Eckhardt, "Temporal Difference Estimation of Dynamic Discrete Choice Models," *Review of Economic Studies*, 2025, *forthcoming.*

Aguirregabiria, Victor and Pedro Mira, "Swapping the nested fixed point algorithm: A class of estimators for discrete Markov decision models," *Econometrica*, 2002, *70*, 1519–1543.

_ and _ , "Dynamic discrete choice structural models: A survey," *Journal of Econometrics*, 2010, *156*, 38–67.

Anderson, Axel, Jeremy Rosen, John Rust, and Kin-Ping Wong, "Disequilibrium Play in Tennis," *Journal of Political Economy*, 2025, *133* (1), 190–251.

Arora, Saurabh and Prashant Doshi, "A survey of inverse reinforcement learning: Challenges, methods and progress," *Artificial Intelligence*, 2021, *297* (103500).

Atashbar, Tokhir and Yunhui Zhao, "AI and Macroeconomic Modeling: Deep Reinforcement Learning in an RBC Model," IMF Working Paper WP/23/040, International Monetary Fund 2023.

Athey, Susan and Guido W. Imbens, "Machine Learning Methods That Economists Should Know About," *Annual Review of Economics*, 2019, *11*, 685–725.

Azar, Nematollah Ab, Aref Shahmansoorian, and Mohsen Davoudi, "From inverse optimal control to inverse reinforcement learning: A historical review," *Annual Reviews in Control*, 2020, *50*, 119–138.

Bajari, Patrick, C. Lanier Benkard, and John Levin, "Estimating Dynamic Models of Imperfect Competition," *Econometrica*, 2007, *75* (5), 1331–1370.

Barnes, M., M.Abueg, O. F. Lange, M. Deeds, J. Trader, D. Molitor, M. Wulfmeier, and S. O'Banion, "Massively Scalable Inverse Reinforcement Learning in Google Maps," *OpenReview.net*, 2024, pp. 1–22.

Barto, A. G., S. J. Bradtke, and S. P. Singh, "Learning to Act with Real Time Dynamic Programming," *Artificial Intelligence*, 1995, *72*, 81–131.

Barzegary, Ebrahim and Hema Yoganarasimhan, "A recursive partitioning approach for dynamic discrete choice modeling in high dimensional settings," *arXiv*, 2022, *arXiv:2208.01476.*

Bayer, Federico A.Bugni Peter Arcidiacono Patrick and Jonathan James, "Approximating High-Dimensional Dynamic Models: Sieve Value Function Iteration," in Eugene Choo and Matthew Shum, eds., *Structural Econometric Models*, Emerald Publishing Ltd., 2013, pp. 44–74.

Bellman, Richard, *Dynamic Programming*, 4th ed., Princeton University Press, 1957.

_ **and Stuart Dreyfus**, "Functional Approximations and Dynamic Programming," *Mathematical Tables and Other Aids to Computation*, 1959, *13* (68), 247–251.

Benitez-Silva, Hugo, George Hall, Hitsch Günter J, Giorgio Pauletto, and John Rust, "A comparison of discrete and parametric approximation methods for continuous-state dynamic programming problems," *manuscript, Yale University*, 2000.

Bertsekas, Dimitri P., *Dynamic Programming and Optimal Control (volumes 1 and 2)*, 4th ed., Athena Scientific, Belmont Massachusetts, 2017.

Cao, Haoyang, Samuel N. Cohen, and Lukasz Szpruch, "Identifiability in inverse reinforcement learning," *arXiv preprint arXiv:2106.03498*, 2021.

Charpentier, Arthur, Romuald Élie, and Carl Remlinger, "Reinforcement Learning in Economics and Finance," *Computational Economics*, 2021, *58*, 433–462.

Chernozhukov, Victor, Juan Carlos Escanciano, Hidehiko Ichimura, Whitney K. Newey, and James M. Robins, "Locally Robust Semiparametric Estimation," *Econometrica*, 2022, *90* (4), 1051–1535.

Christiano, Paul F, Jan Leike, Tom B Brown, Miljan Martic, Shane Legg, and Dario Amodei, "Deep reinforcement learning from human preferences," in "Advances in Neural Information Processing Systems," Vol. 30 2017.

Dearing, Adam and Jason Blevins, "Efficient and Convergent Sequential Pseudo-Likelihood Estimation of Dynamic Discrete Choice Games," *Review of Economic Studies*, 2025, *92*, 981–1021.

Ermon, Stefano, Yexiang Xue, Russell Toth, Bistra Dilkina, Richard Bernstein, Theodoros Damoulas, Patrick Clark, Steve DeGloria, Andrew Mude, Christopher Barrett, and Carla P. Gomes, "Learning large-scale dynamic discrete choice models of spatio-temporal preferences with application to migratory pastoralism in East Africa," *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, 2015, p. 644–650.

Frederick, Shane, George Loewenstein, and Ted O'Donoghue, "Time Discounting and Time Preference: A Critical Review," *Journal of Economic Literature*, 2002, *XL*, 351–401.

Fu, Justin, Aviral Kumar, Matthew Soh, and Sergey Levine, "Diagnosing Bottlenecks in Deep Q-learning Algorithms," in "Proceedings of the 36th International Conference on Machine Learning" PMLR 2019, pp. 2021–2030.

_ **, Katie Luo, and Sergey Levine**, "Learning robust rewards with adversarial inverse reinforcement learning," *Proceedings of ICLR*, 2018, pp. 1—-15.

_ , _ , and _ , "Learning Robust Rewards with Adversarial Inverse Reinforcement Learning," in "International Conference on Learning Representations (ICLR)" 2018.

Fujimoto, Scott, David Meger, Doina Precup, Ofir Nachum, and Shixiang Shane Gu, "Why Should I Trust You, Bellman? The Bellman Error is a Poor Replacement for Value Error," in "Proceedings of the 39th International Conference on Machine Learning" PMLR 2022, pp. 6907–6935.

Garg, Divyansh, Shuvam Chakraborty, Chris Cundy, Jiaming Song, Matthieu Geist, and Stefano Ermon, "IQ-Learn: Inverse soft-Q Learning for Imitation," *arXiv*, 2022, *arXiv:2106.12142v4.*

Ghosal, Gaurav R., Matthew Zurek, Daniel S. Brown, and Anca D. Dragan, "The Effect of Modeling Human Rationality Level on Learning Rewards from Multiple Feedback Types," in "Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence" 2023, pp. 5983–5992.

Gillingham, Kenneth, Fedor Iskhakov, Anders Munk-Nielsen, John Rust, and Bertel Schjerning, "Equilibrium Trade in Automobiles," *Journal of Political Economy*, 2022, *130* (10), 2534–2593.

Gleave, Adam and Sam Toyer, "A Primer on Maximum Causal Entropy Inverse Reinforcement Learning," *arXiv*, 2022, *arXiv:2203.11409v1.*

Haarnoja, Tuomas, Haoran Tang, Pieter Abbeel, and Sergey Levine, "Reinforcement Learning with Deep Energy-Based Policies," *Proceedings of the 34th International Conference on Machine Learning*, 2017.

Ho, Jonathan and Stefano Ermon, "Learning large-scale dynamic discrete choice models of spatio-temporal preferences with application to migratory pastoralism in East Africa," *Advances in neural information processing systems*, 2016, *29.*

Hotz, Joseph V. and Robert A. Miller, "Conditional choice probabilities and the estimation of dynamic models," *Review of Economic Studies*, 1993, *60*, 497–529.

_ , _ , Seth Sanders, and Jeffrey Smith, "A simulation estimator for dynamic models of discrete choice," *Review of Economic Studies*, 1994, *61*, 265–289.

Howard, Ronald A., *Dynamic Programming and Markov Processes*, Technology Press, Cambridge, MA, 1960.

Igami, Mitsuru, "Artificial Intelligence as Structural Estimation: Economic Interpretations of Deep Blue, Bonanza, and AlphaGo," *arXiv preprint arXiv:1710.10967*, 2017.

Imai, Susumu, Neelam Jain, and Andrew Ching, "Bayesian Estimation of Dynamic Discrete Choice Models," *Econometrica*, 2009, *77* (6), 1865–1899.

Iskhakov, Fedor, Jinhyuk Lee, John Rust, Bertel Schjerning, and Kyoungwon Seo, "Constrained Optimization Approaches to Estimation of Structural Models: Comment," *Econometrica*, 2016, *84* (1), 365–370.

_ , **John Rust, and Bertel Schjerning**, "Recursive Lexicographical Search: Finding All Markov Perfect Equilibria of Finite State Directional Dynamic Games," *Review of Economic Studies*, 2015, *83* (2), 658–703.

_ , _ , **and** _ , "Machine Learning and Structural Econometrics: Contrasts and Synergies," *The Econometrics Journal*, 2020, *23* (3), S81–S124.

Jeon, Wonseok, Chen-Yang Su, Paul Barde, Thang Doan, Derek Nowrouzezahrai, and Joelle Pineau, "Regularized Inverse Reinforcement Learning," *OpenReview.net*, 2021, *ICLR2021.*

Jiang, Nan and Tengyang Xie, "Offline Reinforcement Learning in Large State Spaces: Algorithms and Guarantees," *manuscript, University of Illinois*, 2024.

Kalman, R. E., "When is a linear control system optimal?," *Journal of Basic Engineering*, 1964, *86*, 119–138.

Kalouptsidi, Myrto, Paul T. Scott, and Eduardo Souza-Rodrigues, "Identification of counterfactuals in dynamic discrete choice models," *Quantitative Economics*, 2021, *12* (2), 351–403.

Kang, Enoch H., Hema Yoganarasimhan, and Lalit Jain, "An Empirical Risk Minimization Approach for Offline Inverse RL and Dynamic Discrete Choice Model," *arXiv*, 2025, *arXiv:2502.14131.*

Kaufmann, Timo, Paul Weng, Viktor Bengs, and Eyke Hüllermeier, "A Survey of Reinforcement Learning from Human Feedback," *arXiv preprint arXiv:2312.14925*, 2023. Available at https://arxiv.org/abs/2312.14925.

Keane, Michael P., Petra E. Todd, and Kenneth I. Wolpin, "The Structural Estimation of Behavioral Models: Discrete Choice Dynamic Programming Methods and Applications," in Orley Ashenfelter and David Card, eds., *Handbook of Labor Economics*, Vol. 4A, Elsevier, 2011, pp. 331–461.

Khwaja, Ahmed and Sonal Srivastava, "Reinforcement Learning Based Computationally Faster Conditional Choice Simulation Estimation of Dynamic Discrete Choice Models," *manuscript, Judge Business School, University of Cambridge*, 2025.

Kristensen, Dennis, Patrick K. Mogensen, Jong Myun Moon, and Bertel Schjerning, "Solving dynamic discrete choice models using smoothing and sieve methods," *Journal of Econometrics*, 2021, *223*, 328–360.

Lucas, Jr. Robert E., "Econometric Policy Evaluation: A Critique," in Alan Meltzer, ed., *The Phillips Curve and Labor Markets. Carnegie-Rochester Conference Series on Public Policy. Vol. 1*, Elsevier, 1976, pp. 19–46.

Luo, Yao and Peijun Sang, "Efficient Estimation of Structural Models via Sieves," *arXiv*, 2024, *arXiv:2204.13488v2.*

Machina, Mark J., "Choice Under Uncertainty: Problems Solved and Unsolved," *Economic Perspectives*, 1987, *101*, 121–154.

Magnac, Thierry and David Thesmar, "Identifying Dynamic Discrete Decision Processes," *Econometrica*, 2002, *70*, 801–816.

Maskin, Eric and Jean Triole, "Markov Perfect Equilibrium: I. Observable Actions," *Journal of Economic Theory*, 2001, *100*, 191–219.

McFadden, Daniel L., "Conditional Logit Analysis of Qualitative Choice Behavior," in P. Zarembka, ed., *Frontiers in Econometrics*, Academic Press, 1973, pp. 105–142.

Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, 2015, *518*, 518–533.

Mombaur, Katja, Anh Truong, and Jean-Paul Laumond, "From human to humanoid locomotion: an inverse optimal control approach," *Autonomous Robotics*, 2010, *28*, 369–383.

Mullainathan, Sendhil and Jann Spiess, "Machine Learning: An Applied Econometric Approach," *Journal of Economic Perspectives*, 2017, *31* (2), 87–106.

Muth, John F., "Rational Expectations and the Theory of Price Movements," *Econometrica*, 1961, *29* (3), 315–335.

Ng, Andrew Y. and Stuart Russell, "Algorithms for Inverse Reinforcement Learning," *Proceedings of the Seventeenth International Conference on Machine Learning*, 2000, p. 663–670.

_ , Daishi Harada, and Stuart Russell, "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping," in "Proceedings of the Sixteenth International Conference on Machine Learning" 1999, pp. 278–287.

Nguyen, Hoang, "Neural Networks for Efficient Estimation of High-Dimensional Dynamic Discrete Choice Models," *manuscript, Georgetown University*, 2025.

Norets, Andriy, "Inference in Dynamic Discrete Choice Models With Serially orrelated Unobserved State Variables," *Econometrica*, 2009, *77* (5), 1665–1682.

Osa, T., J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters, "An Algorithmic Perspective on Imitation Learning," *Foundations and Trends in Robotics*, 2018, *7*, 1–179.

Ouyang, Long, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray et al., "Training language models to follow instructions with human feedback," in "Advances in Neural Information Processing Systems," Vol. 35 2022, pp. 27730–27744.

Puterman, Martin L., *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, New York: John Wiley & Sons, 1994.

_ and Shelby L. Brumelle, "On the Convergence of Policy Iteration in Stationary Dynamic Programming," *Mathematics of Operations Research*, 1979, *4*, 60–69.

Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn, "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," *arXiv preprint arXiv:2305.18290*, 2023. Available at https://arxiv.org/abs/2305.18290.

Ramachandran, Deepak and Eyal Amir, "Bayesian Inverse Reinforcement Learning," *Proceedings IJCAI*, 2007, p. 2586–2591.

Robbins, H. and S. Munro, "A Stochastic Approximation Method," *Annals of Statistics*, 1951, *22*, 400–407.

Russell, Stuart J. and Peter Norvig, *Artificial Intelligence A Modern Approach*, 4th ed., Pearson, 2022.

Rust, John, "Optimal Replacement of GMC Bus Engines: An Empirical Model of Harold Zurcher," *Econometrica*, 1987, *55* (5), 993–1033.

_ , "Maximum Likelihood Estimation of Discrete Control Processes," *SIAM Journal on Control and Optimization*, 1988, *26* (5), 1006–1024.

_ , "Structural Estimation of Markov Decision Processes," in R. Engle and D. McFadden, eds., *The Handbook of Econometrics, Volume 4*, Elsevier, 1994, pp. 3082–3139.

Samuelson, Paul, "A Note on Measurement of Utility," *Review of Economic Studies*, 1937, pp. 155–161.

Sargent, Thomas J., "Estimation of Dynamic Labor Demand Schedules under Rational Expectations," *Journal of Political Economy*, 1978, *86* (6), 1009–1044.

Silver, David, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, 2018, *362*, 1140–1144.

Simon, Herbert A., *Administrative Behavior: A Study of Decision-Making Processes in Administrative Organization*, 2nd ed., Macmillan, New York, 1957.

Stiennon, Nisan, Long Ouyang, Jeff Wu, Daniel M Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano, "Learning to summarize from human feedback," in "Advances in Neural Information Processing Systems," Vol. 33 2020, pp. 3008–3021.

Su, Che-Lin and Kenneth L. Judd, "Constrained Optimization Approaches to Estimation of Structural Models," *Econometrica*, 2012, *80* (5), 2213–2230.

Sutton, Richard J. and Andrew G. Barto, *Reinforcement Learning An Introduction*, 2nd ed., MIT Press, 2020.

Sutton, Richard S. and Andrew G. Barto, *Reinforcement Learning An Introduction*, 1st ed., MIT Press, 1998.

Tsitsiklis, John N., "Asynchronous Stochastic Approximation and Q-Learning," *Machine Learning*, 1994, *16*, 185–202.

_ **and Benjamin Van Roy**, "An analysis of temporal-difference learning with function approximation," *IEEE transactions on automatic control*, 1997, *42*, 674–690.

van Cranenburgh, Sander, Shenhao Wang, Akshay Vij, Francisco Pereira, and Joan Walker, "Choice Modelling in the Age of Machine Learning," *Journal of Choice Modelling*, 2022, *42*, 100340.

von Neumann, John and Oskar Morgenstern, *The Theory of Games and Economic Behavior*, Princeton University Press, 1944.

Wang, Siyue, Zhaorun Chen, Zhuokai Zhao, Chaoli Mao, Yiyang Zhou, Jiayu He, and Albert Sibo Hu, "ESCIRL: Evolving Self-Contrastive IRL for Trajectory Prediction in Autonomous Driving," *8th Conference on Robot Learning (CoRL 2024)*, 2025, pp. 1–20.

Watkins, C. J. C. H., *Learning from Delayed Rewards*, PhD Thesis, Kings College, Cambridge UK, 1989.

_ **and Peter Dayan**, "Technical Note: Q-Learning," *Machine Learning*, 1992, *8*, 279–282.

Wulfmeier, Markus, Peter Ondrúška, and Ingmar Posner, "Maximum Entropy Deep Inverse Reinforcement Learning," *arXiv*, 2016, *arXiv:1507.04888v3.*

Yuan, Weizhe, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston, "Self-Rewarding Language Models," *arXiv preprint arXiv:2401.10020*, 2024. Available at https://arxiv.org/abs/2401.10020.

Yue, Sheng, Guanbo Wang, Wei Shao, Zhaofeng Zhang, Sen Lin, Ju Ren, and Junshan Zhang, "CLARE: Conservative Model-Based Reward Learning for Offline Inverse Reinforcement Learning," in "The Eleventh International Conference on Learning Representations" 2023.

Zeng, Siliang, Chenliang Li, Alfredo Garcia, and Mingyi Hong, "When Demonstrations Meet Generative World Models: A Maximum Likelihood Framework for Offline Inverse Reinforcement Learning," *37th Conference on Neural Information Processing Systems*, 2023.

_ **, Mingyi Hong, and Alfredo Garcia**, "Structural Estimation of Markov Decision Processes in High-Dimensional State Space with Finite-Time Guarantees," *Operations Research*, 2024, *73* (2).

Zhao, Zhan and Yuebing Liang, "A deep inverse reinforcement learning approach to route choice modeling with context-dependent rewards," *Transportation Research Part C*, 2023, *149.*

Ziebart, Brian D., Andrew Maas, J. Andrew Bagnell, and Anind K. Dey, "Maximum Entropy Inverse Reinforcement Learning," *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, 2008, *55* (5), 1433–1438.

_ **, J. Andrew Bagnell, and Anind K. Dey**, "Modeling Interaction via the Principle of Maximum Causal Entropy," *Proceedings of the 27th International Conference on Machine Learning*, 2010.

Ziegler, Daniel M, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving, "Fine-tuning language models from human preferences," *arXiv preprint arXiv:1909.08593*, 2019.